

بسم تعالی

ماژول های ویراشاپس | SRS

## SRS-01 — User & Account Management

### Software Requirements Specification

Project: Virashops

Module: User & Account Management

Module ID: SRS-01

Version: 1.0

Status: MVP — Scope Baseline

Estimated Development Time: 4 Working Days

Document Type: Detailed Technical Module Specification

Related Scope: MVP Scope & Delivery Plan

#### 1. هدف ماژول

ماژول User & Account Management مسئول ایجاد، مدیریت و کنترل حساب کاربری کاربران ویراشاپس در نسخه MVP است.

این ماژول باید زیرساخت لازم برای:

- ایجاد حساب کاربری
- ورود به سیستم
- مدیریت حساب
- مدیریت پروفایل
- Authentication

- Authorization
- نقش‌ها و دسترسی‌های پایه

را فراهم کند.

این ماژول یکی از اجزای پایه سیستم است و سایر ماژول‌هایی که به شناسایی و سطح دسترسی کاربر نیاز دارند، از آن استفاده خواهند کرد.

Scope رسمی نیز همین چهار حوزه اصلی را برای این ماژول تعیین کرده است.

## 2. محدوده عملکردی MVP

### Registration 2.1

سیستم باید امکان ایجاد حساب کاربری را فراهم کند.

حداقل اطلاعات و رفتارهای موردنیاز:

ایجاد حساب

اعتبارسنجی اطلاعات ورودی

جلوگیری از ایجاد حساب تکراری

ثبت اطلاعات پایه کاربر

ایجاد وضعیت اولیه حساب

نکته: جزئیات دقیق فیلدهای Registration باید با Figma و تصمیم نهایی Product مشخص شود.

### Login 2.2

کاربر باید بتواند وارد حساب خود شود.

سیستم باید:

اطلاعات ورود را دریافت کند.

اعتبار اطلاعات را بررسی کند.

هویت کاربر را احراز کند.

در صورت موفقیت Session/Authentication لازم را ایجاد کند.

در صورت خطا، Response مناسب برگرداند.

جزئیات روش Login که در Scope اولیه مشخص نشده، باید در Technical Design نهایی شود.

## 3. Account Management

کاربر باید بتواند حساب خود را مدیریت کند.

حداقل محدوده: MVP

مشاهده اطلاعات حساب

مشاهده/ویرایش اطلاعات مجاز پروفایل

مدیریت اطلاعات شخصی قابل ویرایش

مشاهده وضعیت حساب

مدیریت اطلاعات مرتبط با حساب در محدوده Figma

هر فیلدی که در Figma نهایی نشده باشد، نباید بدون تأیید Product به Scope قطعی اضافه شود.

#### 4. Profile Management

ماژول باید امکان مدیریت Profile کاربر را فراهم کند.

موارد اصلی:

- مشاهده Profile
- ویرایش اطلاعات Profile
- اعتبارسنجی داده‌ها
- ذخیره تغییرات
- نمایش خطاهای Validation
- جلوگیری از ثبت داده نامعتبر

ساختار دقیق Profile باید با نیاز سایر ماژول‌ها سازگار باشد و از ایجاد وابستگی غیرضروری به UI جلوگیری شود.

#### 5. Authentication

Authentication وظیفه تشخیص هویت کاربر را برعهده دارد.

الزامات:

1. درخواست‌های محافظت‌شده باید دارای وضعیت Authentication معتبر باشند.
2. سیستم باید بتواند User را به درخواست مربوطه متصل کند.
3. درخواست‌های بدون Authentication معتبر نباید به منابع محافظت‌شده دسترسی داشته باشند.
4. وضعیت Authentication باید در Backend کنترل شود و صرفاً به کنترل Frontend وابسته نباشد.
5. خطاهای Authentication باید به شکل استاندارد مدیریت شوند.
6. روش دقیق Token/Session در Scope اولیه تعیین نشده است و باید در Technical Design نهایی ثبت شود.

#### 6. Authorization

Authorization مشخص می‌کند کاربر پس از احراز هویت، به چه منابع و عملیاتی دسترسی دارد.

در MVP، Authorization باید حداقل برای:

User

Seller

Admin

قابل توسعه باشد.

منطق Authorization نباید صرفاً در Frontend قرار گیرد.

Backend باید مرجع نهایی کنترل Permission باشد.

## 7. Basic Roles & Permissions

Scope رسمی این ماژول شامل Basic Roles and Permissions است.

ساختار MVP باید حداقل امکان تعریف Role و نسبت دادن سطح دسترسی مناسب به User را فراهم کند.

نمونه مفهومی:

توجه: این جدول صرفاً ساختار پایه Scope است و Permission های نهایی هر Role باید با Product/Scope سایر ماژول ها نهایی شود.

## 8. Business Rules

### BR-01 — Unique Account

هر حساب باید دارای شناسه قابل تشخیص و یکتا باشد.

### BR-02 — Authentication Required

منابع محافظت شده بدون Authentication معتبر قابل دسترسی نباشند.

### BR-03 — Authorization Required

Authentication به تنهایی به معنی مجاز بودن عملیات نیست.

### BR-04 — Role-Based Access

دسترسی های پایه باید بر اساس Role کنترل شوند.

### BR-05 — Backend Enforcement

کنترل امنیتی اصلی باید در Backend enforce شود.

### BR-06 — Input Validation

اطلاعات ورودی باید قبل از پردازش و ذخیره سازی Validation شوند.

### BR-07 — Error Handling

خطاهای Authentication، Authorization و Validation باید به صورت کنترل شده مدیریت شوند.

## 9. Backend Requirements

Backend این ماژول باید حداقل قابلیت های زیر را فراهم کند:

- Authentication Layer
- Registration
- Login
- Authentication validation
- User identification
- Protected resource access
- Account Layer
- Create user
- Retrieve user
- Update permitted user information
- Account status handling
- Authorization Layer
- Role identification
- Permission checking
- Protected operation enforcement

## 10. API Requirements

API های مربوط به این ماژول باید مطابق معماری پروژه طراحی شوند.

با توجه به Stack اعلام شده برای پروژه، Backend بر پایه NestJS + TypeScript و ارتباط Frontend با Backend از طریق tRPC در نظر گرفته شده است.

API باید:

1. Input Validation داشته باشد.
2. Authentication را کنترل کند.
3. Authorization را enforce کند.
4. Error Response استاندارد داشته باشد.
5. از افشای اطلاعات حساس جلوگیری کند.
6. Contract مشخص و قابل تست داشته باشد.
7. Endpoint/Procedure های دقیق پس از نهایی شدن API Contract ثبت خواهند شد.

## 11. Frontend Requirements

Frontend باید Flow های مربوط به این ماژول را مطابق Figma پیاده سازی کند.

حداقل:

- Registration UI
- Login UI
- Account/Profile UI
- Validation messages
- Loading State
- Error State
- Success State
- Unauthorized State
- Responsive behavior

Frontend باید از API رسمی مازول استفاده کند و نباید منطق امنیتی را صرفاً در Client پیاده‌سازی کند.  
Frontend پروژه بر اساس Scope فعلی با Next.js + React + TypeScript در نظر گرفته شده است.

## 12. UI States

برای Flow های این مازول حداقل State های زیر باید در نظر گرفته شوند:

1. Default
2. فرم آماده استفاده.
3. Loading
4. در حال ارسال/دریافت اطلاعات.
5. Validation Error
6. اطلاعات ورودی نامعتبر است.
7. Authentication Error
8. اطلاعات احراز هویت معتبر نیست.
9. Authorization Error
10. کاربر اجازه انجام عملیات را ندارد.
11. Server Error
12. خطای Backend یا Infrastructure.
13. Success
14. عملیات با موفقیت انجام شده است.
15. Disabled
16. عملیات در شرایط خاص غیرفعال است.

## 13. Security Requirements

امنیت این مازول به دلیل ارتباط مستقیم با هویت کاربران باید جزو بخش‌های اصلی Implementation باشد.  
حداقل الزامات:

- عدم ذخیره اطلاعات حساس به‌صورت ناامن
- Validation ورودی‌ها
- کنترل Authentication در Backend
- کنترل Authorization در Backend

- جلوگیری از دسترسی غیرمجاز
- عدم افشای اطلاعات حساس در Error Response
- مدیریت صحیح Session/Token
- عدم قرار دادن Secret ها در Source Code
- Logging نکردن اطلاعات حساس

Scope کلی پروژه نیز Security را در Definition of Done و بخش Security/QA الزام کرده است.

## 14. Database Requirements

ماژول نیازمند Persistence برای اطلاعات User و Account است. ساختار دقیق Entity ها و Relation ها باید در Data Model پروژه تعریف شود. حداقل مفاهیم داده‌ای:

User

ID

Account information

Profile information

Role/reference

Status

Created At

Updated At

فیلدهای دقیق باید با Data Model نهایی هماهنگ شوند.

در معماری پروژه، PostgreSQL و TypeORM به عنوان Stack Backend اعلام شده‌اند.

## 15. Integration Dependencies

این ماژول با ماژول‌های زیر ارتباط بالقوه دارد:

- Seller Management
- Product & Catalog Management
- Cart & Order Management
- Payment
- Admin & Management Panel
- Notification

به‌خصوص Role و Authorization باید به شکلی طراحی شود که سایر ماژول‌ها بتوانند بدون ایجاد سیستم Permission موازی از آن استفاده کنند.

## 16. Testing Requirements

طبق استاندارد کلی پروژه، تست باید حداقل در سطوح زیر انجام شود:

Unit Test

برای:

Validation

Authentication logic

Authorization logic

Business rules

Integration Test

برای:

Database interaction

Authentication flow

Authorization flow

Integration بین اجزای Backend

E2E Test

برای Flow های اصلی مانند:

Registration → Login → Access Account

و:

Unauthorized Request → Access Denied

Scope کلی پروژه «Unit، Integration، E2E و Manual QA را جزو Definition of Done قرار داده است.

## 17. Error Handling

خطاها باید طبق بندی و مدیریت شوند.

حداقل:

- Invalid Input
- Duplicate Account
- Invalid Credentials
- Unauthorized
- Forbidden
- Resource Not Found



- Server Error

پیام خطا نباید اطلاعات حساس سیستم یا جزئیات داخلی Implementation را افشا کند.

## 18. Logging & Monitoring

رویدادهای مهم سیستم باید قابلیت Logging/Monitoring داشته باشند.

اما:

Secret، Token، Password و اطلاعات حساس Authentication نباید در Log ثبت شوند.

در Stack فعلی، Sentry نیز به عنوان بخشی از Backend/Production Foundation در نظر گرفته شده است.

## 19. Performance

در MVP هدف، Performance مناسب برای عملیات اصلی این مازول است.

Developer باید از:

- Query غیرضروری
- عملیات تکراری
- N+1 Query
- پردازش غیرضروری
- Responseهای بیش از حد بزرگ

اجتناب کند.

Optimization پیشرفته در صورت نیاز و بر اساس داده واقعی انجام خواهد شد.

## 20. Definition of Done — SRS-01

ماژول زمانی Done محسوب می شود که:

- Registration پیاده سازی شده باشد.
- Login پیاده سازی شده باشد.
- Account Management پیاده سازی شده باشد.
- Profile Management پیاده سازی شده باشد.
- Authentication فعال باشد.
- Authorization فعال باشد.
- Basic Roles & Permissions پیاده سازی شده باشد.
- APIهای مورد نیاز تکمیل شده باشند.
- Frontend با Backend Integration شده باشد.
- Validation کامل باشد.
- Error Handling انجام شده باشد.
- Unit Test های لازم نوشته و Pass شده باشند.

- Integration Test های لازم Pass شده باشند.
- E2E Flow های اصلی Pass شده باشند.
- Manual QA انجام شده باشد.
- Security Review انجام شده باشد.
- Code Review انجام شده باشد.
- Documentation مربوط به تغییرات تکمیل شده باشد.
- Build و Lint موفق باشند.

این موارد با Definition of Done کلی پروژه همراستا هستند.

## 21. Acceptance Criteria

AC-01

کاربر جدید می‌تواند طبق Flow تأییدشده حساب ایجاد کند.

AC-02

کاربر ثبت‌شده می‌تواند Login کند.

AC-03

کاربر احراز نشده نمی‌تواند به منابع محافظت‌شده دسترسی پیدا کند.

AC-04

کاربر احراز شده فقط به منابعی که Permission لازم را دارد دسترسی داشته باشد.

AC-05

Role های پایه مطابق Scope قابل استفاده باشند.

AC-06

کاربر بتواند اطلاعات Profile مجاز خود را مشاهده و ویرایش کند.

AC-07

Validation خطاهای ورودی را قبل از ثبت/پردازش تشخیص دهد.

AC-08

خطاهای Authentication و Authorization به صورت کنترل‌شده نمایش داده شوند.

AC-09

اطلاعات حساس در Response و Log افشا نشوند.

AC-10

Flow های اصلی دارای Test مناسب باشند.

AC-11

Frontend مطابق Figma نهایی پیاده‌سازی شود.

AC-12

ماژول پس از Integration با سایر بخش‌های MVP بدون ایجاد Regression قابل استفاده باشد.

## 22. Out of Scope — MVP

موارد زیر صرفاً به دلیل اینکه در Vision کلی ویرایش‌پس وجود دارند، وارد SRS-01 نمی‌شوند مگر اینکه در Scope رسمی MVP اضافه شوند:

- Advanced IAM
- Enterprise SSO
- OAuth provider های متعدد
- Biometric Authentication
- Advanced Risk Engine
- AI Identity/Risk Analysis
- Advanced Trust Score
- پیچیده‌سازی RBAC/ABAC خارج از نیاز MVP
- Multi-tenant Enterprise Identity
- Advanced Fraud Detection

این تفکیک مهم است تا SRS به‌صورت ناخواسته Scope پروژه را افزایش ندهد.

## 23. Dependencies

تکمیل این ماژول وابسته به موارد زیر است:

1. نهایی شدن Figma مربوط به Authentication/Account
2. نهایی شدن API Contract
3. نهایی شدن Role های MVP
4. نهایی شدن اطلاعات Profile
5. آماده بودن Repository و Architecture Foundation
6. آماده بودن Database Environment
7. آماده بودن Development/Testing Environment

## 24. تخمین زمانی

در Scope رسمی:

User & Account Management = 4 Working Days.

این عدد باید شامل چرخه واقعی توسعه باشد و صرفاً زمان Coding تلقی نشود:

Development → Integration → Testing → Debugging → Review

در صورتی که Requirement یا Figma تغییر اساسی داشته باشد، زمان باید طبق Change Request / Scope Change Procedure بررسی شود.

## 25. Developer Deliverables

تحويل SRS-01 باید حداقل شامل:

- Source Code
- Database Changes/Migrations در صورت نیاز
- API/Procedure Implementation
- Frontend Implementation
- Unit Tests
- Integration Tests
- E2E Tests مرتبط
- Documentation
- Technical Notes در صورت وجود تصمیم مهم
- Git Commit/PR
- Deployment/Integration طبق Scope
- رفع Bug های مربوط به Acceptance

باشد.

## 26. معماری و اصل توسعه

این مازول باید مطابق معماری مورد توافق پروژه توسعه داده شود و Business Logic آن نباید به Component های UI وابسته شود.

در Workflow فنی موجود پروژه نیز جریان Backend به صورت:

Requirement → Module → Domain → Use Case → Port → Adapter → API → Tests → Review

تعریف شده است.

بنابراین هدف SRS-01 صرفاً تولید صفحات Login/Register نیست؛ بلکه ایجاد یک Identity/Account Foundation قابل استفاده توسط سایر مازول های MVP است.

## 27. AI-Assisted Development

در صورت استفاده Developer از AI Coding، فرآیند توسعه باید با استاندارد کلی پروژه هماهنگ باشد:

Requirement → Planning → AI Coding → Testing → Review → Debugging → Security Review → Human Approval

کد تولیدشده توسط AI بدون Review انسانی قابل پذیرش نخواهد بود. استاندارد موجود پروژه نیز صراحتاً Human Approval برای AI-generated code را جزو Definition of Done قرار داده است.

## 28. مواردی که قبل از Implementation باید نهایی شوند

موارد زیر در Scope اولیه به صورت جزئی مشخص نشده اند و نباید Developer آنها را بدون تأیید Product Owner قطعی فرض کند:

- روش دقیق Login
- روش دقیق Session/Token
- فیلدهای نهایی Registration
- فیلدهای Profile
- Role های دقیق MVP
- Permission های دقیق هر Role
- سیاست Account Status
- Recovery/Reset Password در صورت نیاز
- Verification در صورت نیاز
- API Contract نهایی
- Figma نهایی

این موارد در زمان Finalization باید به SRS افزوده یا به عنوان تصمیم رسمی Product ثبت شوند.

### وضعیت سند

SRS-01 — User & Account Management

Status: MVP Scope Baseline

Scope: 4 Working Days

Priority: Core / Foundation

Related Module: User & Account Management

Acceptance: مطابق Acceptance Criteria فوق

Source Baseline: MVP Scope & 55-Day Delivery Plan v1.0

### SRS-02 — Seller Management

1. Document Information

Project: Virashops

Module: Seller Management

SRS ID: SRS-02

Version: 1.0

Scope: MVP

Priority: High

Estimated Development Time: 4 Working Days

## Module Purpose .2

ماژول Seller Management مسئول ایجاد، مدیریت و کنترل چرخه اصلی فروشندگان در ویرایش است. این ماژول باید امکان ثبت و مدیریت اطلاعات فروشنده، اطلاعات کسب و کار، وضعیت فروشنده و زیرساخت اولیه Seller Panel را فراهم کند.

## Scope .3

ماژول شامل موارد زیر است:

- Seller Registration
- Seller Profile
- Business Information
- Seller Status Management
- Seller Panel Foundation
- Authentication / Authorization Integration
- ارتباط فروشنده با محصولات و سفارش ها در محدوده MVP

## Seller Registration .4

سیستم باید امکان ایجاد حساب فروشنده را فراهم کند.

اطلاعات پایه مورد نیاز می تواند شامل موارد زیر باشد:

- نام و نام خانوادگی
- شماره موبایل
- Email در صورت نیاز
- اطلاعات کسب و کار
- نام فروشگاه / کسب و کار
- اطلاعات تماس
- اطلاعات پایه مورد نیاز برای فعالیت فروشنده
- ثبت نام باید به سیستم Authentication متصل باشد.

## 5. Seller Profile

هر فروشنده باید دارای Profile مستقل باشد.

اطلاعات Profile شامل:

- Seller ID
- نام فروشنده / کسب و کار
- نام فروشگاه
- اطلاعات تماس
- اطلاعات کسب و کار
- وضعیت حساب
- تاریخ ایجاد حساب
- اطلاعات تکمیلی مورد نیاز MVP

ساختار Profile باید به گونه ای طراحی شود که در آینده قابلیت توسعه داشته باشد.

## 6. Business Information

سیستم باید اطلاعات پایه مربوط به کسب و کار فروشنده را نگهداری کند.

اطلاعات می تواند شامل:

Business Name

Business Type

Contact Information

Address

City / Region

Business Description

سایر اطلاعات ضروری مورد تأیید Product Owner

جزئیات نهایی فیلدها باید مطابق Scope و Figma نهایی شود.

## 7. Seller Status Management

برای هر فروشنده باید Status مشخص وجود داشته باشد.

حداقل وضعیت های مورد نیاز MVP:

Pending

Active

Suspended

Inactive

تغییر وضعیت باید تحت کنترل Authorization انجام شود.

سیستم باید بتواند وضعیت فعلی فروشنده را در بخش‌های مرتبط مانند Seller Panel، Order و Product Management مورد استفاده قرار دهد.

## 8. Seller Panel Foundation

یک زیرساخت اولیه برای Seller Panel باید ایجاد شود.

Seller Panel در MVP باید امکان دسترسی فروشنده به قابلیت‌های مرتبط با حساب خود را فراهم کند.

حداقل ساختار موردنیاز:

- Dashboard Foundation
- Seller Profile
- Business Information
- Seller Status
- دسترسی به محصولات فروشنده
- دسترسی به سفارش‌های مرتبط در محدوده MVP
- Navigation و ساختار قابل توسعه برای Future Features

## 9. Authentication & Authorization

Seller Management باید با سیستم Authentication و Authorization یکپارچه باشد.

الزامات:

1. احراز هویت فروشنده
2. مدیریت Session / Token مطابق معماری پروژه
3. کنترل دسترسی Seller
4. جلوگیری از دسترسی فروشنده به اطلاعات سایر فروشندگان
5. اعمال Role / Permission های تعریف شده
6. اعتبارسنجی درخواست‌ها
7. فروشنده فقط باید بتواند منابع و داده‌هایی را مشاهده یا تغییر دهد که به حساب و سطح دسترسی او مربوط هستند.

## 10. Seller-Product Relationship

سیستم باید ارتباط Seller با Product را پشتیبانی کند.

هر Product یا Seller Listing باید مشخص کند متعلق به کدام فروشنده است.

این ارتباط باید امکان استفاده در موارد زیر را فراهم کند:

- Product Listing
- Product Detail



- Pricing
- Inventory
- Search
- Best Match
- Order
- Seller Panel

## 11. Seller–Order Relationship

سیستم باید امکان ارتباط سفارش با فروشنده را فراهم کند.

اطلاعات مرتبط باید برای استفاده در:

- Order Management
- Seller Panel
- Order Status
- Seller Operations
- Settlement در مراحل آینده

قابل دسترسی باشد.

جزئیات کامل Settlement در Scope مستقل خود تعریف خواهد شد و در این MVP صرفاً Relationship موردنیاز باید ایجاد شود.

## 12. Data Model

حداقل موجودیت اصلی:

Seller

روابط موردنیاز:

Seller → User

Seller → Products

Seller → Orders

ساختار Database باید Modular و قابل توسعه باشد.

فیلدهای دقیق Database باید در طراحی فنی و Implementation نهایی شوند.

## 13. API Requirements

APIهای موردنیاز باید حداقل عملیات زیر را پوشش دهند:

- Seller
- Create Seller

- Get Seller
- Update Seller
- Get Seller Profile
- Update Seller Profile
- Seller Status
- Get Status
- Update Status — Authorized Users Only
- Seller Resources
- Get Seller Products
- Get Seller Orders

APIها باید شامل موارد زیر باشند:

- Authentication
- Authorization
- Validation
- Error Handling
- مناسبترین Response Structure مطابق معماری پروژه

#### **14. Validation**

اطلاعات Seller باید قبل از ذخیره‌سازی Validate شوند.

حداقل موارد:

Required Fields

Valid Data Types

Valid Contact Information

جلوگیری از Duplicate Account در موارد مربوط

Permission Validation

Business Rule Validation

#### **15. Error Handling**

خطاهای مرتبط با Seller Management باید به‌صورت استاندارد مدیریت شوند.

مواردی مانند:

- Seller Not Found
- Unauthorized Access
- Forbidden Action
- Invalid Data

- Duplicate Seller
- Invalid Status Change
- Internal Server Error

باید Response مناسب داشته باشند.

## 16. Security Requirements

الزامات امنیتی:

- Authentication اجباری برای عملیات خصوصی
- Authorization برای عملیات مدیریتی
- جلوگیری از IDOR
- جلوگیری از دسترسی Seller به داده Seller دیگر
- Input Validation
- کنترل دسترسی API
- ثبت خطاهای امنیتی مهم
- عدم افشای اطلاعات حساس در API Response

## 17. Frontend / UI Requirements

UI مربوط به Seller Management باید مطابق Figma نهایی پیاده‌سازی شود.

حداقل صفحات / بخش‌ها:

- Seller Registration
- Seller Profile
- Business Information
- Seller Dashboard Foundation
- Seller Status
- Seller Products
- Seller Orders
- UI باید:
- Responsive باشد.
- Loading State داشته باشد.
- Empty State داشته باشد.
- Error State داشته باشد.
- Success State داشته باشد.
- Disabled State داشته باشد.

## 18. Integration Requirements

Seller Management باید با ماژول‌های زیر Integration داشته باشد:

- User & Account Management
- Product & Catalog Management
- B2C Retail
- B2B / Wholesale
- Cart & Order Management
- Admin & Management Panel
- API & Backend Integration
- Notification

Integration باید از طریق Contract های مشخص و معماری Modular انجام شود.

## 19. Admin Integration

Admin باید بتواند در محدوده MVP:

1. Seller ها را مشاهده کند.
2. اطلاعات Seller را مشاهده کند.
3. وضعیت Seller را مدیریت کند.
4. Seller های فعال / غیرفعال / معلق را مشاهده کند.
5. اطلاعات پایه Seller را مدیریت کند.
6. سطح دسترسی Admin باید مطابق Role و Permission های سیستم باشد.

## 20. Notification Integration

در صورت وجود Event های مرتبط، Seller Management باید امکان ایجاد Notification را برای Notification Module فراهم کند.

نمونه Event ها:

Seller Registration

Seller Activation

Seller Suspension

Status Change

پیاده سازی کانال های Notification مطابق Scope ماژول Notification انجام می شود.

## 21. Testing Requirements

حداقل تست های مورد انتظار:

Unit Tests

برای Business Logic های اصلی.

Integration Tests

برای:

Seller/User Relationship

Seller/Product Relationship

Seller/Order Relationship

API

Authorization

E2E Tests

برای Flow های اصلی:

Seller Registration

Seller Login

Seller Profile

Seller Status

Seller Resource Access

## 22. Performance Requirements

ماژول باید از ابتدا با ساختار قابل توسعه پیاده‌سازی شود.

موارد مورد انتظار:

1. Query های بهینه
2. Pagination در List ها
3. جلوگیری از Query های غیرضروری
4. Index های مناسب
5. استفاده صحیح از Cache در صورت نیاز
6. عدم ایجاد وابستگی غیرضروری بین Modules

## 23. Documentation

موارد زیر باید مستند شوند:

- Module Structure
- Data Model
- API Contract
- Authentication / Authorization Rules
- Business Rules

- Important Technical Decisions
- Integration Points

## 24. Definition of Done

ماژول زمانی Complete محسوب می‌شود که:

- Seller Registration پیاده‌سازی شده باشد.
- Seller Profile پیاده‌سازی شده باشد.
- Business Information پیاده‌سازی شده باشد.
- Seller Status Management پیاده‌سازی شده باشد.
- Seller Panel Foundation آماده باشد.
- Authentication / Authorization Integration انجام شده باشد.
- Seller/Product Relationship پیاده‌سازی شده باشد.
- Seller/Order Relationship پیاده‌سازی شده باشد.
- APIهای موردنیاز پیاده‌سازی شده باشند.
- Validation و Error Handling تکمیل شده باشد.
- Security Baseline رعایت شده باشد.
- Unit / Integration / E2E Tests موردنیاز اجرا شده باشند.
- Frontend مطابق Figma پیاده‌سازی شده باشد.
- Responsive States بررسی شده باشند.
- Integration با ماژول‌های مرتبط انجام شده باشد.
- Code Review انجام شده باشد.
- Documentation تکمیل شده باشد.
- Deployment در Environment مربوطه انجام شده باشد.
- Product Owner Review انجام شده باشد.
- Acceptance دریافت شده باشد.

## 25. Out of Scope — MVP

موارد زیر در این SRS جزو MVP محسوب نمی‌شوند مگر اینکه در Scope نهایی به‌صورت صریح اضافه شوند:

- Advanced Seller Analytics
- Advanced Seller CRM
- Seller Subscription Plans
- Advanced Seller Advertising
- Advanced Seller AI Assistant
- Advanced Seller Credit
- Advanced Seller Financing
- Advanced Seller Settlement
- Advanced Seller Rating Intelligence
- Advanced Seller Fraud Detection
- AI Seller Optimization

این موارد می‌توانند در نسخه‌های بعدی توسعه داده شوند.

## **Estimated Development Time .26**

Target: 4 Working Days

این زمان بر اساس Scope فعلی MVP است و مشروط به آماده بودن Figma، تصمیمات Product، محیط توسعه و وابستگی‌های فنی است.

## **SRS-03 — Product Catalog Management**

Document Information .1

Document Title: Software Requirements Specification — Product Catalog Management

Document ID: VIRA-SRS-03

Module: Product Catalog Management

Project: Virashops

Version: 1.0

Status: MVP Requirement Specification

Priority: High

Audience: Product Owner, CTO, Backend Developer, Frontend Developer, UI/UX, QA

Related Modules: Seller Management, Category Management, Order Management, Admin & Management Panel, Search, Pricing, Inventory

## **Purpose .2**

این سند الزامات نرم‌افزاری ماژول Product Catalog Management ویرایش را برای نسخه MVP مشخص می‌کند. هدف ماژول، ایجاد و مدیریت ساختار استاندارد کالاها و اطلاعات مربوط به آنها در سیستم است؛ به‌گونه‌ای که محصولات بتوانند توسط فروشندگان ایجاد، تکمیل، مدیریت و برای نمایش به مشتری منتشر شوند. این سند مبنای طراحی، توسعه، تست، بررسی و پذیرش ماژول Product Catalog Management خواهد بود.

## **Scope .3**

ماژول Product Catalog Management در MVP باید امکان مدیریت چرخه اصلی اطلاعات محصول را فراهم کند، از جمله:

ایجاد محصول

ویرایش محصول

مشاهده محصول

مدیریت اطلاعات پایه

اتصال محصول به دسته‌بندی

مدیریت تصاویر

مدیریت قیمت

مدیریت موجودی

مدیریت وضعیت انتشار

مدیریت ویژگی‌های محصول در محدوده تعریف‌شده MVP

اتصال محصول به فروشنده

نمایش اطلاعات محصول در بخش‌های مرتبط سیستم

این ماژول باید به‌صورت Modular طراحی شود تا در نسخه‌های آینده امکان توسعه به سمت PIM، Product Intelligence، AI Product Enrichment، Dynamic Pricing و Advanced Catalog Management وجود داشته باشد.

#### 4. User Roles

##### 4.1 Seller

فروشنده می‌تواند محصولات متعلق به خود را:

ایجاد کند.

ویرایش کند.

مشاهده کند.

قیمت و موجودی را مدیریت کند.

تصاویر محصول را مدیریت کند.

وضعیت انتشار محصول خود را مشاهده کند.

##### 4.2 Admin

مدیر سیستم می‌تواند:

محصولات را مشاهده کند.

محصولات را ویرایش یا مدیریت کند.

وضعیت انتشار را کنترل کند.



محصولات نامعتبر را غیرفعال کند.

اطلاعات محصول را در صورت نیاز اصلاح یا مدیریت کند.

### Customer 4.3

مشتری در MVP عمدتاً مصرف‌کننده اطلاعات Catalog است و می‌تواند:

محصولات منتشرشده را مشاهده کند.

اطلاعات محصول را مشاهده کند.

قیمت و موجودی را مشاهده کند.

تصاویر و مشخصات محصول را مشاهده کند.

محصول را به Cart اضافه کند.

### Product Entity .5

هر Product باید دارای یک شناسه یکتا باشد.

حداقل اطلاعات موردنیاز:

Product ID

Seller ID

Product Name

SKU در صورت استفاده

Category ID

Brand در صورت نیاز MVP

Short Description

Description

Product Images

Price

Discount در صورت فعال بودن

Stock Quantity

Product Status

Created At

Updated At

ساختار نهایی Entity و نام فیلدها توسط CTO در Data Model مشخص می‌شود.

## Product Creation .6

سیستم باید امکان ایجاد محصول جدید را برای Seller مجاز فراهم کند.

فرآیند ایجاد:

Seller وارد بخش محصولات شود.

گزینه ایجاد محصول جدید را انتخاب کند.

اطلاعات پایه محصول را وارد کند.

دسته‌بندی را انتخاب کند.

تصاویر را اضافه کند.

قیمت را وارد کند.

موجودی را وارد کند.

اطلاعات را ثبت کند.

سیستم Validation انجام دهد.

محصول ایجاد شود.

محصول در وضعیت مناسب MVP قرار گیرد.

سیستم نباید اجازه ایجاد محصول با اطلاعات ضروری ناقص را بدهد.

## Product Editing .7

Seller مجاز باید بتواند محصول متعلق به خود را ویرایش کند.

قابل ویرایش بودن فیلدها باید بر اساس Role و Business Rules تعیین شود.

حداقل موارد:

نام

توضیحات

تصاویر

دسته‌بندی

قیمت

موجودی

ویژگی‌های قابل مدیریت

تمام تغییرات باید با Updated At ثبت شوند.

## **Product Ownership .8**

هر Product باید به Seller مربوطه متصل باشد.

Seller نباید بتواند محصول متعلق به Seller دیگر را مشاهده یا تغییر دهد، مگر در صورت داشتن Permission مناسب.

Authorization باید در Backend اعمال شود و صرفاً به محدودیت UI وابسته نباشد.

## **Category Assignment .9**

هر محصول باید به Category مناسب متصل شود.

سیستم باید:

Category معتبر را دریافت کند.

امکان انتخاب Category را فراهم کند.

از اتصال محصول به Category نامعتبر جلوگیری کند.

در صورت نیاز امکان تغییر Category را فراهم کند.

ساختار Category Management در SRS مربوط به Category Management تعریف می‌شود و Product Catalog صرفاً از آن استفاده می‌کند.

## **Product Images .10**

سیستم باید امکان مدیریت تصاویر محصول را فراهم کند.

حداقل قابلیت‌ها:

Upload Image

حذف تصویر

انتخاب تصویر اصلی

مدیریت چند تصویر

نمایش Preview

اعتبارسنجی فایل

محدودیت‌های زیر باید قابل تعریف باشند:

نوع فایل

حجم فایل

تعداد تصاویر

ابعاد مجاز در صورت نیاز

Media Architecture و Storage توسط معماری فنی سیستم تعیین می‌شود.

## 11. Product Pricing

هر محصول باید حداقل یک قیمت قابل فروش داشته باشد.

سیستم باید امکان:

ثبت قیمت

ویرایش قیمت

نمایش قیمت

ثبت Discount در صورت وجود

محاسبه قیمت نهایی طبق Pricing Rules

را فراهم کند.

منطق پیشرفته Pricing، Tiered Pricing، Dynamic Pricing و Group Buying Pricing در ماژول‌های مربوطه تعریف خواهد شد و نباید بدون تعریف Business Rule وارد این ماژول شود.

## 12. Inventory

محصول باید دارای وضعیت موجودی باشد.

حداقل اطلاعات:

Stock Quantity

Stock Status

نمونه وضعیت‌ها:

In Stock

Low Stock

Out of Stock

موجودی باید در Backend کنترل شود.

Frontend نباید منبع نهایی حقیقت موجودی باشد.

در نسخه‌های بعدی، Inventory می‌تواند به:

Multi-Warehouse

Reservation

Real-Time Inventory

Supplier Inventory

Distributed Inventory

توسعه پیدا کند.

### Product Status .13

Product باید دارای Lifecycle Status باشد.

حداقل وضعیت های MVP:

- Draft
- Active
- Inactive
- Out of Stock
- در صورت نیاز Business Rules می توانند وضعیت های دیگری مانند:
- Pending Review
- Rejected
- Archived
- را اضافه کنند.

وضعیت نهایی و Transition های مجاز باید توسط CTO و Product Owner مشخص شود.

### Product Publishing .14

محصول تنها زمانی باید برای Customer قابل مشاهده باشد که شرایط انتشار آن برقرار باشد.

حداقل شروط پیشنهادی:

- Product معتبر باشد.
- Seller مجاز باشد.
- Category معتبر باشد.
- قیمت معتبر باشد.
- وضعیت محصول اجازه نمایش بدهد.

در صورت نیاز، Admin می تواند Product را فعال یا غیرفعال کند.

### Product Listing .15

Seller باید بتواند لیست محصولات متعلق به خود را مشاهده کند.

حداقل اطلاعات Listing:

Product Name

Image

SKU در صورت استفاده

Price

Stock

Category

Status

Updated At

قابلیت‌های موردنیاز:

Search

Filter

Sort

Pagination

## 16. Product Search

سیستم باید امکان جستجوی Product را فراهم کند.

در MVP جستجو می‌تواند بر اساس:

Product Name

SKU

Category

باشد.

معماری باید به‌گونه‌ای باشد که در آینده امکان اتصال به Search Engine تخصصی فراهم باشد.

Search پیشرفته، Semantic Search و AI Search در Scope فعلی این SRS نیستند.

## 17. Product Validation

سیستم باید هنگام Create و Update محصول Validation انجام دهد.

موارد نمونه:

نام خالی نباشد.

قیمت معتبر باشد.

موجودی مقدار معتبر داشته باشد.

Category معتبر باشد.

Seller مجاز باشد.

فایل‌های تصویری معتبر باشند.

داده‌های غیرمجاز رد شوند.

Validation باید در Backend انجام شود.

Frontend Validation صرفاً برای بهبود UX است و جایگزین Backend Validation نیست.

## 18. Authorization

تمام عملیات Product باید Permission-based باشد.

نمونه:

جزئیات Permission Matrix در Security/Authorization Architecture تعیین می‌شود.

## 19. API Requirements

ماژول باید API‌های موردنیاز برای Frontend و سایر سرویس‌های سیستم ارائه کند.

نمونه Endpoint‌های منطقی:

- Create Product
- Get Product
- Update Product
- Delete/Deactivate Product
- List Products
- Search Products
- Update Stock
- Update Price
- Upload Product Image
- Remove Product Image
- Publish Product
- Unpublish Product

نامگذاری نهایی API‌ها توسط CTO تعیین می‌شود.

API‌ها باید:

- Authenticated باشند.
- Authorization داشته باشند.
- Input Validation داشته باشند.
- Error Handling استاندارد داشته باشند.
- Response Structure مشخص داشته باشند.

## 20. Database Requirements

Product Catalog باید در Database دارای ساختار قابل توسعه باشد.

حداقل موجودیت‌های منطقی:

- Product
- ProductImage
- ProductCategory Relation در صورت نیاز
- ProductPrice در صورت نیاز
- ProductInventory در صورت نیاز
- ساختار نهایی Database توسط CTO طراحی می‌شود.
- Business Logic نباید به ساختار UI یا Figma وابسته باشد.

## 21. Integration Requirements

Product Catalog باید قابلیت Integration با ماژول‌های زیر را داشته باشد:

- Seller Management
- برای تعیین مالک محصول.
- Category Management
- برای اتصال محصول به دسته‌بندی.
- Inventory
- برای کنترل موجودی.
- Pricing
- برای قیمت‌گذاری.
- Cart
- برای افزودن Product به سبد خرید.
- Order Management
- برای ثبت Product در Order.
- Search
- برای جستجوی Product.
- Admin Panel
- برای مدیریت محصولات.
- Media/Storage
- برای تصاویر و فایل‌های محصول.

## 22. Cart Integration

Customer باید بتواند Product فعال و قابل فروش را به Cart اضافه کند.

قبل از اضافه شدن به Cart باید حداقل موارد زیر بررسی شود:



- Product موجود باشد.
- Product Active باشد.
- Seller معتبر باشد.
- Stock کافی باشد.
- Price معتبر باشد.
- قیمت نهایی نباید صرفاً از داده ارسال شده توسط Client پذیرفته شود.

## 23. Order Integration

در زمان ایجاد Order، اطلاعات Product باید از Backend و منابع معتبر سیستم دریافت شود. Order نباید صرفاً به اطلاعات فعلی Frontend اعتماد کند. اطلاعات مورد نیاز می تواند شامل:

Product ID

Seller ID

Unit Price

Quantity

Product Snapshot

باشد.

ثبت Snapshot در Order برای حفظ اطلاعات تاریخی سفارش توصیه می شود.

## 24. Security Requirements

سیستم باید در برابر موارد زیر محافظت شود:

- Unauthorized Product Access
- Unauthorized Product Modification
- Price Manipulation
- Stock Manipulation
- Invalid File Upload
- Injection Attacks
- Broken Access Control
- Mass Assignment
- Malicious Input

تمام عملیات حساس باید در Backend کنترل شوند.

## 25. Performance Requirements

Product Listing و Product Detail باید برای استفاده معمول MVP عملکرد مناسبی داشته باشند.

موارد مورد انتظار:

Pagination برای Listing های بزرگ

جلوگیری از Query های غیرضروری

Index مناسب Database

استفاده منطقی از Cache در صورت نیاز

جلوگیری از N+1 Query

بهینه سازی تصاویر

مقادیر دقیق Performance SLA در Technical Architecture یا Performance Specification تعیین می شود.

## 26. Error Handling

سیستم باید خطاهای قابل پیش بینی را به شکل استاندارد مدیریت کند.

نمونه:

- Product Not Found
- Unauthorized Access
- Invalid Product Data
- Invalid Category
- Invalid Price
- Insufficient Stock
- Invalid Image
- Upload Failed
- Product Already Inactive

پیام خطا نباید اطلاعات حساس سیستم را افشا کند.

## 27. Auditability

عملیات مهم Product باید قابلیت Trace داشته باشند.

حداقل:

Created By

Created At

Updated By در صورت نیاز

Updated At

Status Changes در صورت نیاز

در آینده امکان ایجاد Audit Log کامل‌تر وجود خواهد داشت.

## 28. UI/UX Requirements

UI باید مطابق Figma تأییدشده پیاده‌سازی شود.

صفحات اصلی MVP می‌تواند شامل:

- Product List
- Add Product
- Edit Product
- Product Detail
- Product Images
- Product Pricing
- Product Inventory
- Product Status

باشد.

UI باید:

- Responsive باشد.
- Mobile-friendly باشد.
- Loading State داشته باشد.
- Empty State داشته باشد.
- Error State داشته باشد.
- Success Feedback داشته باشد.
- Validation Message داشته باشد.

## 29. Required UI States

حداقل State‌های موردنیاز:

- Default
- Loading
- Empty
- Error
- Success
- Disabled
- Unauthorized
- Not Found
- Out of Stock
- Inactive

### **Non-Functional Requirements .30**

ماژول باید دارای ویژگی‌های زیر باشد:

Maintainability

کد باید قابل نگهداری و توسعه باشد.

Scalability

ساختار باید امکان افزایش تعداد Product و Seller را فراهم کند.

Security

دسترسی‌ها و داده‌های حساس باید در Backend محافظت شوند.

Testability

Business Logic باید قابل تست باشد.

Modularity

ماژول باید مرزبندی مشخص با سایر Domain‌ها داشته باشد.

Portability

Business Logic نباید به Framework یا UI خاصی وابستگی غیرضروری داشته باشد.

Documentation

API و منطق مهم سیستم باید مستند باشد.

### **Testing Requirements .31**

Developer باید حداقل موارد زیر را تست کند:

Unit Test

برای Business Logic اصلی.

Integration Test

برای ارتباط Product با:

Database

Seller

Category

Inventory

Pricing

API Test

برای Endpoint های اصلی.

Authorization Test

برای جلوگیری از دسترسی Seller به محصولات سایر فروشندگان.

Validation Test

برای داده های نامعتبر.

Edge Case Test

از جمله:

Stock = 0

Price = 0

Product بدون تصویر

Category حذف شده

Seller غیر فعال

Product غیر فعال

Quantity بیشتر از Stock

## 32. Acceptance Criteria

Product Catalog Management زمانی قابل پذیرش است که:

- Seller بتواند Product ایجاد کند.
- Product دارای اطلاعات پایه معتبر باشد.
- Product به Seller صحیح متصل شود.
- Product به Category صحیح متصل شود.
- Product قابل ویرایش باشد.
- Product Images قابل مدیریت باشند.
- Price قابل مدیریت باشد.
- Inventory قابل مدیریت باشد.
- Product Status قابل مدیریت باشد.
- Product Listing صحیح کار کند.
- Search/Filter/Sort در محدوده MVP کار کند.
- Authorization صحیح اعمال شود.
- Product فعال برای Customer قابل مشاهده باشد.
- Product غیر فعال برای Customer نمایش داده نشود.
- Product بتواند به Cart متصل شود.
- Product اطلاعات صحیح را به Order منتقل کند.
- Validation در Backend انجام شود.

- خطاها به صورت استاندارد مدیریت شوند.
- تست‌های تعریف‌شده موفق باشند.
- UI مطابق Figma تأییدشده پیاده‌سازی شود.
- Responsive/Mobile behavior مطابق Scope طراحی باشد.
- کد در Repository پروژه ثبت شود.
- Documentation موردنیاز تحویل شود.
- Deployment طبق فرآیند پروژه انجام شود.
- Product Owner بتواند مازول را بررسی و تأیید کند.

### 33. Out of Scope — MVP

موارد زیر در این SRS جزو MVP نیستند، مگر اینکه در Scope نهایی پروژه صراحتاً اضافه شوند:

AI Product Enrichment

AI Product Generation

Dynamic Pricing

Advanced PIM

Semantic Product Search

AI Search

Product Recommendation Engine

Advanced Product Bundling

Advanced Multi-Warehouse Catalog

Automated Supplier Catalog Import

Advanced Product Intelligence

Dynamic Group Buying Pricing

این موارد می‌توانند در نسخه‌های بعدی توسعه پیدا کنند.

### 34. Future Extensibility

معماری Product Catalog باید امکان توسعه آینده به سمت موارد زیر را داشته باشد:

- Product Variants
- Product Attributes
- Product Specifications
- Brand Management
- Product Bundles
- Product Kits

- Multi-Seller Offers
- Multi-Warehouse Inventory
- Product Import
- Supplier Catalog
- PIM
- AI Product Enrichment
- AI Categorization
- AI Recommendation
- Semantic Search
- Price Intelligence
- Dynamic Pricing
- Group Buying

این قابلیت‌ها نباید بدون Scope و Requirement مستقل وارد MVP شوند.

### 35. Technical Constraints

1. Business Logic باید مستقل از UI باشد.
2. API Contract باید مشخص و مستند باشد.
3. Authorization باید در Backend enforce شود.
4. Client نباید منبع نهایی قیمت یا موجودی باشد.
5. Database باید قابلیت توسعه داشته باشد.
6. ماژول باید با معماری کلی Virashops سازگار باشد.
7. توسعه نباید مانع انتقال یا توسعه آینده به معماری اختصاصی Virashops شود.
8. وابستگی غیرضروری به یک Framework یا UI Component خاص باید کاهش یابد.
9. تغییرات مهم Architecture باید با CTO و Product Owner هماهنگ شود.

### 36. Deliverables

Developer باید موارد زیر را تحویل دهد:

- Source Code
- Database/Migration Changes
- API Implementation
- API Documentation
- Unit Tests
- Integration Tests در محدوده توافق شده
- Configuration Changes در صورت نیاز
- Technical Notes
- Deployment Changes در صورت نیاز
- Documentation مربوط به Business Logic مهم

تحویل صرفاً شامل کد نوشته شده نیست و باید مطابق Delivery & Acceptance Specification پروژه انجام شود.

### **Traceability .37**

تمام Requirements این SRS باید قابلیت اتصال به موارد زیر را داشته باشند:

Feature Scope → SRS Requirement → Figma/UI → Implementation → Test → Review → Acceptance

هر Requirement مهم باید قابل ردیابی تا مرحله تست و پذیرش باشد.

این ساختار با اصول SRS حرفه‌ای که بر Requirements قابل تست، Interface، Security، Performance، Qualification و Traceability تأکید می‌کنند هم‌راستا است.

### **Change Control .38**

هر Requirement خارج از این SRS که در طول توسعه اضافه شود، باید از طریق فرآیند Change Request / Scope Change Procedure بررسی شود.

هیچ قابلیت جدیدی صرفاً بر اساس درخواست شفاهی نباید به‌عنوان تعهد توسعه‌ای محسوب شود.

### **Final Acceptance .39**

پس از تکمیل توسعه:

Development → Testing → Integration → Deployment → Product Review → Acceptance

ماژول پس از تأیید Product Owner و تکمیل موارد توافق‌شده، به‌عنوان Accepted ثبت خواهد شد.

## **SRS-04 — B2C RETAIL**

### **Document Information .1**

Document Title: B2C Retail Management — Software Requirements Specification

Document Type: Module-Level SRS

Project: Virashops

Module ID: SRS-04

Module Name: B2C Retail

Version: 1.0

Status: MVP

Priority: Must Have



## **Module Overview .2**

B2C Retail module is responsible for providing the standard retail purchasing flow for end customers.

The module enables a customer to:

- Browse products
- View product details
- Select the required quantity
- Add products to cart
- Review cart contents
- Proceed to checkout
- Complete a standard retail purchase

The module must provide a simple customer-facing purchasing experience while integrating with the Product Catalog, Cart & Order Management, Payment, User & Account, Notification, and API/Backend Integration modules.

The current MVP scope identifies B2C Retail as a core module with an estimated development allocation of 4 working days.

## **Objectives .3**

The objectives of this module are:

- Provide a complete standard B2C purchasing journey.
  - Allow customers to discover and select products.
  - Provide accurate product and pricing information.
- Allow customers to manage selected products through Cart.
  - Provide a clear Checkout process.
- Connect the purchase flow to Order Management.
  - Connect the purchase flow to Payment.
- Provide appropriate loading, error and success states.
  - Maintain secure authorization and data handling.
- Provide a responsive customer experience.

## **Scope .4**

#### In Scope 4.1

The MVP B2C Retail module includes:

Product Listing

Product Detail

Quantity selection

Add to Cart

Cart interaction

Checkout

Standard retail purchase flow

Integration with Order Management

Integration with Payment

Customer authentication where required

Validation

Error handling

Responsive UI

Required API integration

The supplied MVP scope explicitly defines these B2C capabilities as:

Product Listing

Product Detail

Cart

Checkout

Standard retail purchase

#### **Out of Scope .5**

The following capabilities are not part of the core B2C MVP module unless explicitly approved through the project's Scope Change Procedure:

Group Buying

AI Shopping Assistant

Voice Commerce

Dynamic Pricing

Advanced recommendation engine

Advanced loyalty system

Credit/Installment purchasing

Check-based purchasing

Advanced financing

Subscription commerce

Advanced personalization

Native mobile application

These capabilities may be introduced in later phases.

## **User Roles .6**

The primary actor is:

Customer

A registered customer who intends to purchase products through the B2C retail flow.

Depending on the final authentication/business rules, guest users may be permitted to browse products and potentially initiate checkout, but final behavior must follow the approved product requirements and authentication rules.

## **Functional Requirements .7**

FR-01 — Product Listing

The system shall provide a product listing interface.

The customer shall be able to:

View available products

View product image

View product name

View product price

View basic product information

Select a product to open its detail page

Product Listing must use the Product & Catalog Management module as the source of product information.

#### FR-02 — Product Detail

The system shall provide a Product Detail page.

The page shall display, where applicable:

Product name

Product images

Product price

Available quantity

Product attributes

Product description

Seller information where required

Purchase quantity

Add to Cart action

Product availability and pricing must be retrieved from the backend rather than being treated as static frontend data.

#### FR-03 — Quantity Selection

The customer shall be able to select the desired quantity.

The system shall validate:

Minimum purchasable quantity

Maximum purchasable quantity

Available inventory

Product-specific quantity restrictions

The frontend shall not be considered the authoritative source for inventory validation.

Final validation must occur on the backend.

#### FR-04 — Add to Cart

The customer shall be able to add a product to the Cart.

The system shall:

Validate the product.

Validate availability.

Validate quantity.

Calculate the applicable product price.

Add/update the cart item.

Return the updated cart state.

Cart persistence and business rules belong to the Cart & Order Management module.

#### FR-05 — Cart

The B2C flow shall provide access to the customer's Cart.

The Cart shall display:

Product

Quantity

Unit price

Item subtotal

Cart total

Available actions

Customer actions shall include:

Increase quantity

Decrease quantity

Remove item

Continue shopping

Proceed to Checkout

The B2C module shall consume the Cart functionality rather than duplicating Cart business logic.

#### **Checkout .8**

#### FR-06 — Checkout Initiation

The customer shall be able to proceed from Cart to Checkout.

Before checkout proceeds, the system shall validate:

Cart is not empty

Products still exist

Products are purchasable

Prices are valid

Inventory is available

Required customer information is available

Required delivery information is available

#### FR-07 — Checkout Information

Checkout shall collect or display the information required for completing the retail purchase.

Depending on the final approved business rules, this may include:

Customer information

Delivery information

Address

Contact information

Order summary

Payment method

Delivery/shipping information

Only fields required by the final MVP business rules should be implemented.

#### FR-08 — Order Summary

Before payment, the customer shall be shown a final order summary containing, as applicable:

Products

Quantities

Unit prices

Subtotal

Shipping/delivery cost

Discounts

Applicable fees

Final payable amount

The customer must be able to review the order before initiating payment.

## **Payment Integration .9**

### **FR-09 — Payment Initiation**

The B2C module shall initiate payment through the Payment module.

The B2C module must not implement independent payment business logic.

The payment process shall follow the centralized Payment module.

The MVP Payment scope includes:

Payment gateway

Transactions

Success/failed states

Callback

Payment status

Error handling

## **Order Creation .10**

### **FR-10 — Order Creation**

A successful checkout process shall result in an Order being created through the Cart & Order Management module.

The B2C module shall not create a parallel order system.

The standard flow shall be conceptually:

Product → Cart → Checkout → Payment → Order → Order Status

The exact transaction/order sequencing shall follow the approved backend architecture.

## **Error Handling .11**

The system shall support appropriate error states, including:

Product unavailable

Product out of stock

Invalid quantity

Price changed

Cart expired/invalid

Checkout validation failure

Payment failure

Payment timeout

Network/API failure

Order creation failure

Errors must be presented clearly to the customer without exposing sensitive technical information.

## **UI States .12**

The B2C interface shall support at minimum:

Default

Loading

Empty

Error

Disabled

Out of Stock

Success

Payment Pending

Payment Failed

Payment Success

The exact visual implementation shall follow the approved Figma designs.

## **Responsive Requirements .13**

The B2C module must be responsive.

The customer-facing purchasing journey shall support:



Desktop

Tablet

Mobile

The mobile experience must maintain the complete purchasing flow without requiring desktop-specific functionality.

The frontend technology proposal identifies Next.js, React and TypeScript as the customer-facing application stack, with Tailwind CSS and reusable UI primitives available for implementation.

### **Backend Integration .14**

The module shall communicate with the backend through the project's approved API layer.

The current technical direction identifies:

NestJS

TypeScript

PostgreSQL

TypeORM

Redis

S3/MinIO

Next.js

React

TypeScript

tRPC

as part of the MVP technology stack.

The frontend shall not directly access the database.

### **Validation Requirements .15**

Validation must exist at appropriate layers.

Frontend Validation

For:

Required fields

Quantity format

Basic input constraints

User feedback

Backend Validation

For:

Product availability

Inventory

Pricing

Authorization

Cart ownership

Order ownership

Payment state

Business rules

Backend validation is authoritative.

## **Security Requirements .16**

The module shall respect the project's authentication and authorization architecture.

Security requirements include:

Customer ownership validation

Prevention of unauthorized Cart access

Prevention of unauthorized Order access

Server-side price validation

Server-side inventory validation

Input validation

Protection against manipulation of product/price identifiers

Secure payment integration

No exposure of secrets

Appropriate rate limiting where required

Security-critical functionality must be tested according to the project's security standards.

## **API Requirements .17**

The final API contract shall be defined by the backend implementation.

The module is expected to require API capabilities corresponding to:

Product retrieval

Product detail

Cart retrieval

Cart modification

Checkout validation

Order creation

Payment initiation

Payment status

Customer information where required

API contracts must be documented and version-controlled.

## **Data Requirements .18**

The B2C module shall consume data from existing domain modules rather than creating duplicate entities.

Primary dependencies include:

User

Product

Category

Inventory

Cart

Cart Item

Order

Order Item

Payment

The exact database schema is governed by the project's centralized database architecture.

## **Module Dependencies .19**

The B2C Retail module depends primarily on:

User & Account Management — 01

For customer identity and authentication.

Product & Catalog Management — 03

For product, pricing, inventory and media information.

Search & Discovery — 06

For product discovery where applicable.

Cart & Order Management — 07

For Cart and Order functionality.

Payment — 08

For payment processing.

Notification — 10

For relevant purchase/payment notifications.

API & Backend Integration — 11

For communication between frontend and backend.

The MVP scope explicitly separates these capabilities into the corresponding modules.

## **Architecture Requirements .20**

The B2C module must follow the project's architectural principles.

The current technical direction is:

Modular Monolith + Hexagonal Architecture + DDD Lite

The module must maintain clear separation between:

Domain

Application / Use Cases

Ports

Adapters

API

Infrastructure

Domain logic must not become dependent on frontend/UI implementation.

The project's development workflow specifies the sequence:

Requirement → Module → Domain → Use Case → Port → Adapter → API → Tests → Review

## **Testing Requirements .21**

The B2C module shall include appropriate:

Unit Tests

For:

Quantity rules

Cart-related use cases

Checkout validation

Business rules

Integration Tests

For:

Database interactions

Cart persistence

Order creation

Payment integration boundaries

E2E Tests

At minimum, the primary customer journey should be tested:

Product → Cart → Checkout → Payment → Order

Manual QA

Manual verification shall cover:

Desktop

Mobile

Loading states

Error states

Payment states

Inventory edge cases

Customer ownership/security scenarios

The project's technical workflow requires Unit → Integration → E2E → Docker/runtime testing where applicable.

## **Performance Requirements .22**

The B2C module should avoid unnecessary client-side processing and duplicate API calls.

Performance considerations include:

Efficient product loading

Appropriate caching

Optimized images

Pagination where required

Efficient Cart retrieval

Avoidance of unnecessary database queries

Appropriate loading states

The exact performance targets should be finalized in the technical implementation specification.

## **Definition of Done .23**

The B2C module is considered technically complete when:

Product Listing is implemented.

Product Detail is implemented.

Quantity selection works.

Add to Cart works.

Cart integration is complete.

Checkout is implemented.

Order integration is complete.

Payment integration is complete.

Required validation is implemented.

Error states are implemented.

Responsive UI is implemented according to approved Figma.

Required APIs are integrated.

Unit/integration/E2E tests appropriate to the feature are completed.

Security review is completed for relevant flows.

Code is committed to the repository.

Deployment/staging verification is completed.

Product Owner can execute and verify the complete B2C purchasing journey.

#### **Acceptance Criteria .24**

The module shall be accepted when the Product Owner can successfully execute the following scenario:

Customer opens a product → views product information → selects quantity → adds product to Cart → reviews Cart → proceeds to Checkout → completes required information → initiates payment → payment is processed → Order is created/updated correctly → customer receives the appropriate result/status.

No critical or blocking defect may remain in the primary B2C purchasing flow.

The project's MVP acceptance baseline also requires Figma-aligned implementation, responsive behavior, tested purchase flow and absence of critical bugs.

#### **Scope Boundary .25**

The B2C Retail module must remain within the approved MVP scope.

Any request introducing:

New payment models

Installment/check-based purchasing

Group Buying

Advanced recommendation

AI

Voice

Subscription

Loyalty

Major changes to Checkout

Major changes to Figma

New external integrations

must be evaluated through the project's Change Request / Scope Change Procedure.

The current 55-day MVP delivery plan explicitly states that new features, major Figma changes, new integrations or materially expanded edge cases may require a revised estimate.

## **Future Extension Points .26**

The B2C module should be designed so that future capabilities can be added without rewriting the core purchasing flow.

Potential future extensions include:

Group Buying

Multi-price purchasing

Installment purchasing

Check-based purchasing

Credit purchasing

AI Shopping Assistant

Best Match

Personalized recommendations

Voice Commerce

Dynamic Pricing

Loyalty

Promotional campaigns

Advanced fulfillment models

These capabilities are not part of the current B2C MVP implementation unless separately approved.

## **Technical Documentation .27**

The implementation shall be documented sufficiently to allow future developers to understand:

Module purpose

Responsibilities



Non-responsibilities

Domain rules

Dependencies

API contracts

Persistence requirements

Security considerations

Test coverage

Acceptance criteria

The project's technical documentation standard specifically calls for module documentation covering purpose, responsibilities/non-responsibilities, domain rules, dependencies, security, API, persistence, test matrix and acceptance.

## **Final Module Boundary .28**

The B2C Retail module is the standard retail purchasing experience of Virashops.

Its core responsibility is:

Product Discovery/Selection → Cart → Checkout → Standard Purchase

It must integrate with the centralized Product, Cart, Order and Payment domains and must not create parallel business logic for those systems.

MVP Development Allocation: 4 Working Days.

## **SRS-05 — B2B عمده‌فروشی**

### **1. اطلاعات سند**

عنوان سند: مشخصات نیازمندی‌های نرم‌افزاری مدیریت فروش عمده B2B

شناسه سند: SRS-05

پروژه: Virashops

ماژول: B2B Wholesale

نسخه: 1.0

وضعیت: Scope نسخه MVP

مخاطبان: QA ،UI/UX ،Frontend ،Backend ،CTO ،Product Owner

## 2. معرفی ماژول

ماژول B2B Wholesale مسئول پشتیبانی از فرآیند خرید و فروش عمده در ویرایشگاه است. این ماژول باید به خریداران سازمانی و تجاری اجازه دهد محصولات عمده را مشاهده کنند، شرایط خرید عمده را بررسی کنند، مقدار موردنیاز را انتخاب کنند، سفارش عمده ایجاد کنند و سفارش‌های خود را مدیریت کنند.

ماژول باید به‌عنوان بخشی از معماری اصلی تجارت ویرایشگاه پیاده‌سازی شود و با زنجیره استاندارد زیر سازگار باشد:

Product → Cart → Order → Payment → Fulfillment → Shipment → Settlement

ماژول B2B Wholesale نباید یک سیستم سفارش موازی و مستقل ایجاد کند.

## 3. اهداف ماژول

اهداف اصلی:

1. فراهم‌کردن امکان مشاهده محصولات عمده.
2. پشتیبانی از قیمت‌گذاری عمده.
3. پشتیبانی از خرید بر اساس مقدار.
4. پشتیبانی از حداقل و حداکثر مقدار خرید.
5. پشتیبانی از ضرب فروش.
6. ایجاد سفارش عمده.
7. مدیریت سفارش توسط خریدار B2B.
8. مدیریت سفارش عمده توسط فروشنده.
9. اتصال به Product ،Cart ،Order ،Payment و Fulfillment.
10. فراهم‌کردن زیرساخت مناسب برای توسعه قابلیت‌های پیشرفته B2B در آینده.

## 4. نقش‌های کاربری

### 4.1 خریدار B2B

شخص یا سازمانی که با هدف تجاری یا سازمانی از ویرایشگاه خرید می‌کند.

نمونه‌ها:

فروشگاه‌های خرده‌فروشی

رستوران‌ها

هتل‌ها

شرکت‌ها

دفاتر

توزیع‌کنندگان

سایر سازمان‌ها و مشتریان تجاری

#### 4.2 فروشنده

فروشنده‌ای که محصولات خود را با شرایط عمده در ویرایش‌پس عرضه می‌کند.

#### 4.3 مدیر سیستم

مدیر سیستم که مسئول مدیریت و نظارت بر عملیات B2B Wholesale است.

### 5. محصول عمده

هر محصول می‌تواند علاوه بر فروش عادی، قابلیت فروش عمده نیز داشته باشد.

سیستم باید بتواند بین موارد زیر تفاوت ایجاد کند:

قابلیت فروش خرده

قابلیت فروش عمده

برای محصولی که فروش عمده آن فعال است، شرایط خرید عمده باید در سیستم ذخیره و در اختیار بخش‌های مرتبط قرار گیرد.

### 6. قیمت‌گذاری عمده

سیستم باید امکان تعریف قیمت عمده توسط فروشنده را داشته باشد.

اطلاعات قیمت‌گذاری عمده می‌تواند شامل موارد زیر باشد:

- قیمت واحد عمده
- حداقل مقدار خرید
- حداکثر مقدار خرید
- مضرب خرید
- موجودی قابل فروش
- وضعیت فعال/غیرفعال بودن فروش عمده

سیستم باید مبلغ سفارش را بر اساس مقدار انتخاب شده و قوانین قیمت گذاری مربوطه محاسبه کند.

## 7. حداقل مقدار خرید

برای هر محصول عمده، فروشنده می تواند حداقل مقدار خرید را تعیین کند.

مثال:

حداقل خرید عمده: ۱۰ عدد

سیستم نباید اجازه ثبت سفارش عمده کمتر از مقدار تعیین شده را بدهد.

## 8. حداکثر مقدار خرید

فروشنده می تواند حداکثر مقدار قابل خرید در یک سفارش عمده را مشخص کند.

سیستم باید مقدار انتخاب شده را قبل از Checkout و ایجاد سفارش بررسی کند.

## 9. مضرب خرید

در صورت نیاز، فروشنده می تواند مضرب خرید تعیین کند.

مثال:

حداقل خرید: ۱۲ عدد

مضرب خرید: ۱۲ عدد

مقادیر معتبر:

۱۲

۲۴

۳۶

۴۸

مقادیر نامعتبر:

۱۳

۲۰

سیستم باید این محدودیت را در زمان انتخاب مقدار و همچنین در Backend بررسی کند.

## 10. اطلاعات محصول عمده

صفحه محصولی که قابلیت فروش عمده دارد باید اطلاعات مربوط به خرید عمده را نمایش دهد، از جمله:

- نام محصول
- تصویر محصول
- مشخصات محصول
- قیمت عمده
- حداقل مقدار خرید
- حداکثر مقدار خرید در صورت وجود
- مضرب خرید در صورت وجود
- موجودی
- اطلاعات فروشنده
- شرایط ارسال
- شرایط پرداخت در صورت وجود

## 11. انتخاب مقدار

خریدار B2B باید بتواند مقدار موردنیاز خود را انتخاب کند.

رابط کاربری می‌تواند شامل موارد زیر باشد:

- فیلد ورود تعداد
- دکمه افزایش/کاهش
- نمایش حداقل مقدار
- نمایش حداکثر مقدار
- نمایش مضرب خرید
- اعتبارسنجی مقدار

در صورت واردکردن مقدار نامعتبر، سیستم باید دلیل نامعتبر بودن مقدار را به‌صورت واضح به کاربر نمایش دهد.

## 12. اتصال به Cart

محصولات عمده باید با Cart اصلی ویرایش‌سازگار باشند.

Cart باید موارد زیر را محاسبه کند:

- تعداد
- قیمت واحد
- مبلغ هر آیتم
- تخفیف‌های قابل اعمال
- هزینه ارسال در صورت وجود
- کارمزدهای قابل اعمال
- مبلغ نهایی

قوانین خرید عمده باید قبل از ورود به Checkout مجدداً اعتبارسنجی شوند.

### 13. ایجاد سفارش

پس از تکمیل موفق Checkout، سیستم باید یک Order استاندارد ویرایش ایجاد کند.

سفارش باید حداقل اطلاعات زیر را حفظ کند:

- خریدار
- فروشنده
- محصول
- مقدار
- قیمت واحد
- قانون قیمت عمده
- مبلغ کل
- وضعیت پرداخت

## SRS-06 — Research, Search & Product Discovery

سند نیازمندی‌های نرم‌افزاری (Software Requirements Specification))

### 1. معرفی ماژول

ماژول Research / Search / Discovery مسئول ایجاد امکان کشف، جستجو، بررسی و پیدا کردن کالاها و اطلاعات مرتبط در ویرایش است.

هدف این ماژول آن است که کاربر بتواند با کمترین اصطکاک، نیاز خود را بیان کرده و به کالا، فروشنده، قیمت و گزینه‌های مناسب دسترسی پیدا کند.

این ماژول در معماری آینده ویرایش می‌تواند پایه‌ای برای قابلیت‌های پیشرفته‌تر مانند Recommendation، Best Match، AI، Shopping Assistant، Semantic Search و Conversational Commerce باشد.

در طراحی Catalog و Search باید اطلاعات ساختاریافته کالا، دسته‌بندی، ویژگی‌ها، Variant، قیمت و وضعیت موجودی به شکلی استاندارد در دسترس موتور Discovery قرار گیرد.

## 2. اهداف ماژول

اهداف اصلی:

- فراهم کردن جستجوی سریع کالا.
- نمایش نتایج مرتبط با عبارت جستجو.
- امکان Browse و کشف کالاها از طریق Category.
- امکان Filter و Sort نتایج.
- نمایش اطلاعات اصلی کالا در نتایج.
- هدایت کاربر از Search Result به Product Detail.
- ثبت رفتارهای مهم کاربر برای Analytics.
- ایجاد زیرساخت مناسب برای توسعه Search و Discovery پیشرفته در آینده.
- آمادهسازی معماری برای Semantic Search و AI-powered Discovery در مراحل بعدی.

## 3. محدوده MVP

### Search 3.1

کاربر باید بتواند از طریق Search Box عبارت موردنظر خود را وارد کند.

نمونه:

یخچال

تلویزیون سامسونگ

موتور

لپتاپ

یخچال ۳۰ فوت

سیستم باید نتایج مرتبط را نمایش دهد.

Search می‌تواند در MVP بر اساس اطلاعات ساختاریافته Product مانند:

نام محصول

Brand

Category

SKU

بخشی از Description

Attributes

انجام شود.

#### 4. Search Suggestion

در صورت وجود زیرساخت لازم، سیستم می‌تواند هنگام تایپ کاربر پیشنهادهایی نمایش دهد.

مثلاً:

یخچال

پیشنهادهای:

یخچال ساید

یخچال دوقلو

یخچال سامسونگ

یخچال ال‌جی

یخچال ۳۰ فوت

این بخش در MVP می‌تواند در سطح پایه پیاده‌سازی شود و قابلیت‌های هوشمندتر در Future توسعه پیدا کنند.

#### 5. Search Result

صفحه Search Result باید حداقل شامل موارد زیر باشد:

- Search Query
- تعداد نتایج
- Product Card
- تصویر محصول
- نام محصول
- قیمت
- وضعیت موجودی
- فروشنده در صورت نیاز
- اطلاعات اصلی محصول
- امکان ورود به Product Detail

#### 6. Category Discovery

کاربر باید بتواند بدون استفاده از Search نیز محصولات را از طریق Category ها پیدا کند.

ساختار Category باید امکان Hierarchical Browsing را داشته باشد.

مثلاً:

لوازم خانگی → یخچال → یخچال ساید

و یا:

دیجیتال → موبایل → گوشی سامسونگ



ساختار Category باید با Product Catalog هماهنگ باشد تا Search، Filter و Discovery بتوانند از اطلاعات یکسان استفاده کنند.

## 7. Filtering

در MVP امکان Filter بر اساس ویژگی‌های اصلی محصول در نظر گرفته شود.

نمونه:

- Category
- Brand
- Price Range
- Availability
- Seller
- Product Attributes

Filterها باید Dynamic باشند و بر اساس Category و Attributes قابل توسعه باشند.

ویژگی‌هایی که برای مشتری مهم هستند باید در ساختار داده به‌صورت قابل جستجو و Filterable ذخیره شوند، نه صرفاً به‌صورت متن آزاد در Description.

## 8. Sorting

در MVP امکان Sort بر اساس موارد اصلی در نظر گرفته شود:

مرتبطترین

ارزانترین

گرانترین

جدیدترین

در مراحل بعد امکان توسعه به:

محبوبترین

بیشترین فروش

بیشترین تخفیف

بهترین ارزش خرید

Best Match

وجود داشته باشد.

## 9. Product Discovery

Discovery صرفاً Search نیست.

سیستم باید امکان کشف کالا از مسیرهای مختلف را فراهم کند:

- Search
- Category
- Filter
- Related Products
- Popular Products
- Recently Viewed
- پیشنهادهای آینده
- Group Buying در صورت فعال بودن

هدف این است که کاربر حتی اگر نام دقیق محصول را نداند، بتواند به گزینه مناسب برسد.

## 10. Product Card

Product Card مورد استفاده در Search و Discovery باید قابلیت نمایش اطلاعات اصلی زیر را داشته باشد:

- Product Image
- Product Name
- Brand
- Price
- Discount
- Availability
- Seller
- Rating در صورت وجود
- Group Buying Indicator در صورت وجود
- سایر Badges مورد نیاز

ساختار Card باید Component-based باشد تا در بخشهای مختلف سیستم قابل استفاده مجدد باشد.

## 11. Search Query Handling

سیستم باید در MVP از موارد پایه زیر پشتیبانی کند:

- Partial Match
- Keyword Match
- نام محصول
- Brand
- Category
- SKU در صورت نیاز
- عبارتهای چندکلمه‌ای
- در نسخه‌های بعدی قابلیت‌های زیر قابل اضافه شدن هستند:
- Synonym

- Typo Tolerance
- Fuzzy Search
- Semantic Search
- Natural Language Query
- Intent Detection

Semantic Search در آینده می‌تواند امکان درک Intent کاربر و ارتباط مفهومی بین عبارت جستجو و اطلاعات محصول را فراهم کند.

## 12. Empty Search Result

اگر نتیجه‌ای پیدا نشد، سیستم نباید صرفاً صفحه خالی نمایش دهد.

باید پیام مناسب نمایش داده شود:

- نتیجه‌ای برای جستجوی شما پیدا نشد.
- و در صورت امکان پیشنهادهایی مانند:
- بررسی املای عبارت
- حذف Filter
- جستجوی عبارت مشابه
- نمایش Category مرتبط
- محصولات مشابه

ارائه شود.

## 13. Search Analytics

رفتارهای مهم کاربر باید از ابتدا Event Tracking داشته باشند.

حداقل Event ها:

- Search Performed
- Search Result Viewed
- Search Result Clicked
- Filter Applied
- Filter Removed
- Sort Changed
- Product Viewed
- Search With Zero Result
- Add To Cart From Search

این داده‌ها در آینده برای بهبود Search، Ranking و Recommendation استفاده خواهند شد. سیستم‌های Discovery نیز برای بهینه‌سازی نتایج به داده‌های تعامل کاربر مانند Impression، Click و Purchase وابسته هستند.

## 14. Search Ranking

در MVP Ranking می‌تواند ساده و Rule-based باشد.

معیارهای اولیه:

Relevance

Product Availability

Category Match

Keyword Match

در مراحل بعد Ranking می‌تواند از عوامل بیشتری استفاده کند:

Popularity

Sales

Conversion Rate

Seller Quality

Price

Stock

User Preferences

Location

Purchase History

Behaviour

Business Rules

## 15. ارتباط با Product Catalog

Search و Discovery نباید دارای Catalog مستقل و ناسازگار با Product Catalog باشند.

Product Catalog باید Source اصلی اطلاعات محصول باشد و Search Index در صورت استفاده از Search Engine، نمایه‌ای از داده‌های Catalog باشد.

تغییرات مهم مانند:

Product Created

Product Updated

Price Updated

Inventory Updated

Product Published

## Product Unpublished

باید در نهایت باعث بهروزرسانی Search Index شوند.

در معماری Search، جداسازی Product Record از Search Index می‌تواند برای Scalability و Performance مناسب باشد و در عوض نیازمند مدیریت Consistency و Index Synchronization است.

## 16. Search Index

در صورت استفاده از Search Engine اختصاصی، معماری باید قابلیت استفاده از ابزارهایی مانند:

Elasticsearch

OpenSearch

یا سرویس مشابه

را داشته باشد.

انتخاب نهایی Search Engine بر عهده Developer/CTO و بر اساس:

حجم Catalog

Query Volume

Performance

Cost

Scalability

Complexity

خواهد بود.

این مورد الزام به استفاده از یک تکنولوژی مشخص در MVP نیست.

## 17. API Requirements

Backend باید API‌های موردنیاز برای Search و Discovery را فراهم کند.

نمونه Endpoint‌های مفهومی:

- GET /search
- GET /search/suggestions
- GET /categories
- GET /categories/{id}/products
- GET /products/{id}
- GET /products/{id}/related

نامگذاری و ساختار نهایی API باید مطابق معماری Backend پروژه تعیین شود.

## 18. Performance Requirements

Search یکی از مسیرهای حساس سیستم است.

هدف معماری باید:

- Response سریع
- Pagination
- Efficient Querying
- Indexing
- Caching در صورت نیاز
- جلوگیری از Query های سنگین
- قابلیت Scale شدن

باشد.

در نسخه های آینده می توان SLA مشخص برای Search تعریف کرد.

## 19. Pagination

Search Result باید از Pagination پشتیبانی کند.

روش نهایی می تواند بر اساس معماری انتخابی باشد:

Offset Pagination

Cursor Pagination

یا روش مناسب Search Engine

در نسخه های بزرگ، روش Pagination باید از نظر Performance بررسی شود.

## 20. Mobile UX

Search و Discovery باید Mobile-first طراحی شود.

در Mobile حداقل موارد زیر باید ساده و سریع باشند:

- Search
- Search Suggestion
- Filter
- Sort
- Product Card
- Product Detail Navigation

Filter می تواند به صورت Bottom Sheet یا Full-screen Filter طراحی شود.

## Desktop UX .21

در Desktop:

- Search Bar
- Suggestion
- Sidebar Filter
- Sort
- Product Grid/List
- Pagination

باید به صورت Responsive طراحی شوند.

## Authentication .22

Search پایه نباید الزاماً نیازمند Login باشد.

کاربر Guest نیز باید بتواند:

- Search
- Browse
- Filter
- View Product
- را انجام دهد.
- اما قابلیت‌هایی مانند:
- Personalized Discovery
- Search History
- Recommendation
- Saved Search

در آینده می‌توانند به User Account متصل شوند.

## Security .23

سیستم باید در برابر موارد زیر مقاوم باشد:

- Injection
- Malformed Queries
- Abuse
- Search Spam
- Excessive Requests
- Scraping در حد نیاز
- Rate Limit Abuse

Rate Limiting می‌تواند در API Gateway یا لایه مناسب Backend اعمال شود.

## 24. Future Discovery Architecture

معماری باید قابلیت توسعه به سیستم هوشمند Discovery را داشته باشد.

- Phase 2
- Synonym
- Typo Tolerance
- Advanced Ranking
- Search Analytics
- Related Products
- Popularity
- Personalization پایه
- Phase 3
- Semantic Search
- Intent Detection
- AI Recommendation
- Natural Language Search
- Conversational Search
- AI Shopping Assistant
- Best Match Engine

## 25. ارتباط با AI

در آینده کاربر می‌تواند به جای Search سنتی بگوید:

«یک یخچال برای خانواده ۴ نفره تا ۸۰ میلیون می‌خواهم.»

سیستم باید بتواند:

- Intent را تشخیص دهد.
- Budget را استخراج کند.
- Product Category را تشخیص دهد.
- ویژگی‌های موردنیاز را استخراج کند.
- محصولات مناسب را پیدا کند.
- قیمت و شرایط فروشندگان را مقایسه کند.
- Best Match را ارائه دهد.

این بخش خارج از Scope اصلی MVP است اما معماری Discovery باید امکان توسعه آن را حفظ کند.

## 26. ارتباط با Best Match



در آینده Search می‌تواند یکی از Data Source های Best Match Engine باشد.

Best Match می‌تواند علاوه بر Relevance از موارد زیر استفاده کند:

- Price
- Total Cost
- Seller
- Location
- Delivery
- Payment Terms
- Availability
- Customer Preferences
- Seller Reputation

در نتیجه خروجی صرفاً «مرتبطترین محصول» نخواهد بود، بلکه می‌تواند بهترین گزینه برای همان مشتری باشد.

## 27. ارتباط با Group Buying

در صورت وجود Group Buying، Search و Discovery باید بتوانند Group Order های فعال را نیز در آینده نمایش دهند.  
مثلاً:

یخچال X

محصولات معمولی

خرید گروهی فعال

خرید گروهی نزدیک به تکمیل

این بخش در MVP تنها در صورت قرار گرفتن Group Buying در Scope اجرایی فعال می‌شود.

## 28. Out of Scope — MVP

موارد زیر در MVP این SRS الزاماً پیاده‌سازی نمی‌شوند:

- AI Search
- Semantic Search
- Conversational Search
- Voice Search
- AI Recommendation
- AI Shopping Assistant
- Dynamic Ranking مبتنی بر Machine Learning
- Predictive Search پیشرفته
- AI Group Formation

این موارد در Roadmap آینده قرار می‌گیرند.

## 29. Acceptance Criteria

ماژول زمانی قابل پذیرش است که:

- کاربر بتواند Product را Search کند.
- نتایج مرتبط نمایش داده شوند.
- Search Result به Product Detail متصل باشد.
- Category Browsing کار کند.
- Filterهای تعریف شده کار کنند.
- Sorting کار کند.
- Empty Result State وجود داشته باشد.
- Product Card استاندارد باشد.
- Pagination یا مکانیزم مناسب نمایش نتایج پیاده‌سازی شود.
- Eventهای اصلی Search ثبت شوند.
- APIهای موردنیاز Frontend و Backend Integration شوند.
- Error Handling انجام شده باشد.
- Responsive UX در Mobile و Desktop تست شده باشد.
- عملکرد Search در حجم داده مورد توافق قابل قبول باشد.
- Code Review و Testing انجام شده باشد.
- Documentation فنی مرتبط تحویل شود.

## 30. Developer Deliverables

Developer باید در پایان توسعه ارائه کند:

- Source Code
- Search APIs
- Search/Discovery Components
- Database/Search Schema در صورت نیاز
- Search Index Configuration در صورت استفاده
- Integration با Product Catalog
- Analytics Events
- Unit Tests
- Integration Tests در بخش‌های لازم
- Error Handling
- Technical Documentation
- Deployment Configuration

## 31. وابستگی‌های ماژول

این ماژول با بخش‌های زیر ارتباط دارد:

Product Catalog Management

Category Management

Seller Management

Inventory

Pricing

Product Detail

User Management

Analytics

Recommendation

Group Buying

Best Match

AI Services

برخی از این وابستگی‌ها در MVP ممکن است صرفاً Read-only یا API-level باشند.

## 32. اصل معماری

Search/Discovery نباید به UI یا Framework خاصی وابسته شود.

Business Logic مربوط به:

- Search
- Filtering
- Ranking
- Discovery Rules
- Search Analytics
- باید در Backend و Service Layer قابل مدیریت باشد.
- Frontend صرفاً باید Consumer این قابلیت‌ها باشد.

این ساختار باعث می‌شود در آینده امکان توسعه، تغییر Frontend یا انتقال به معماری تخصصی ویرایش بدون بازنویسی کامل Business Logic وجود داشته باشد.

## 33. Future Vision

هدف نهایی این مازول تبدیل Search از یک ابزار ساده جستجو به یک Intelligent Product Discovery Engine است.

مسیر تکامل:

Keyword Search



Advanced Search



Semantic Search



Personalized Discovery



Best Match



AI Shopping Assistant



Conversational Commerce



Voice + AI + Dynamic UI

در این معماری، Search فقط وظیفه پیدا کردن کالا را ندارد؛ بلکه به یکی از لایه‌های اصلی سیستم تصمیم‌گیری خرید ویراشاپس تبدیل خواهد شد.

## SRS-07 — Cart & Order Management

سند نیازمندی‌های نرم‌افزاری

### 1. مشخصات سند

نام پروژه: Virashops

نام ماژول: Cart & Order Management

کد: SRS-07

نسخه: 1.0

وضعیت: MVP

نوع سند: Software Requirements Specification

مخاطبان: Product Owner، CTO، Backend، Frontend، QA، UI/UX

اولویت: Critical / Must Have

## 2. هدف سند

این سند نیازمندی‌های ماژول Cart & Order Management ویرایش‌پس را برای نسخه MVP تعریف می‌کند.

هدف ماژول این است که کاربر بتواند محصولات انتخاب‌شده را در سبد خرید مدیریت کرده و پس از بررسی نهایی، سفارش خود را ایجاد و وضعیت آن را مشاهده و پیگیری کند.

این ماژول باید به‌صورت مستقیم با زنجیره اصلی Commerce ویرایش‌پس یکپارچه باشد:

Product → Cart → Order → Payment → Fulfillment → Shipment → Settlement

Cart و Order نباید به‌عنوان یک سیستم مستقل از معماری اصلی Commerce پیاده‌سازی شوند.

ساختار SRS بر مبنای تفکیک Purpose، Scope، Functional Requirements، Interfaces، Business Rules و Non-Functional Requirements تنظیم شده است؛ این ساختار با الگوهای رایج SRS برای سیستم‌های Commerce نیز هم‌راستا است.

## 3. محدوده ماژول

### 3.1 موارد داخل Scope

ماژول MVP شامل موارد زیر است:

- ایجاد Cart
- مشاهده Cart
- افزودن محصول به Cart
- حذف محصول از Cart
- تغییر Quantity
- بررسی موجودی
- اعتبارسنجی قیمت
- محاسبه Subtotal
- محاسبه Discount در صورت وجود
- محاسبه هزینه‌های قابل اعمال
- محاسبه Total
- انتقال Cart به Checkout / Order Creation
- ایجاد Order
- ایجاد Order Item
- تخصیص شماره سفارش
- ثبت اطلاعات مشتری
- ثبت اطلاعات تحویل
- ثبت قیمت نهایی در لحظه ایجاد سفارش
- ثبت وضعیت سفارش
- مشاهده جزئیات سفارش
- مشاهده تاریخچه سفارش‌ها
- مدیریت وضعیت‌های اصلی Order
- اتصال Order به Payment
- اتصال Order به Fulfillment

- مدیریت خطاهای ایجاد سفارش
- Notification پایه مرتبط با Order

#### 4. موارد خارج از Scope MVP

موارد زیر در این SRS به عنوان قابلیت های Future در نظر گرفته می شوند، مگر اینکه در Scope نهایی پروژه به صورت رسمی اضافه شوند:

- AI Order Optimization
- AI Cart Assistant
- Smart Cart Recommendation
- Dynamic Pricing
- Advanced Order Splitting
- Advanced Group Buying
- Advanced Subscription Order
- Advanced Recurring Order
- Intelligent Logistics Optimization
- Advanced Fraud Detection
- Predictive Order Cancellation
- AI-based Order Recovery
- Advanced Multi-Warehouse Optimization
- Advanced Marketplace Order Matching

#### 5. کاربران و نقش ها

##### Customer 5.1

کاربر مشتری می تواند:

محصول به Cart اضافه کند.

Cart را مشاهده کند.

Quantity را تغییر دهد.

محصول را حذف کند.

Checkout را آغاز کند.

Order ایجاد کند.

Order های قبلی خود را مشاهده کند.

وضعیت Order را مشاهده کند.

اطلاعات Order را مشاهده کند.

##### Seller 5.2

Seller در این ماژول مستقیماً Cart را مدیریت نمی‌کند، اما اطلاعات Orderهای مرتبط با محصولات او باید در اختیار سیستم Seller Management قرار گیرد.

### Admin 5.3

Admin باید بتواند در سطح موردنیاز MVP:

Orderها را مشاهده کند.

وضعیت Order را مشاهده کند.

در صورت داشتن Permission مناسب، وضعیت Order را مدیریت کند.

خطاهای Order را بررسی کند.

## 6. Cart Management

### 6.1 ایجاد Cart

سیستم باید بتواند برای Customer یک Cart فعال ایجاد یا Cart فعال موجود را بازیابی کند.

Cart باید دارای شناسه یکتا باشد.

### 6.2 افزودن محصول به Cart

کاربر باید بتواند محصول معتبر را به Cart اضافه کند.

اطلاعات حداقلی:

Product ID

Variant ID در صورت وجود

Seller ID در صورت نیاز

Quantity

Unit Price

Cart ID

### 6.3 اعتبارسنجی محصول

قبل از افزودن محصول، سیستم باید حداقل موارد زیر را بررسی کند:

محصول وجود دارد.

محصول فعال است.

محصول قابل خرید است.

Seller در وضعیت قابل فروش است.

موجودی قابل قبول است.

Quantity معتبر است.

محدودیت Minimum Quantity رعایت شده است.

محدودیت Maximum Quantity رعایت شده است.

Multiples / Pack Size در صورت وجود رعایت شده است.

## 7. تغییر Quantity

کاربر باید بتواند Quantity هر Cart Item را افزایش یا کاهش دهد.

سیستم باید پس از هر تغییر:

موجودی را بررسی کند.

محدودیت‌های خرید را بررسی کند.

قیمت را در صورت نیاز Recalculate کند.

Subtotal را به‌روزرسانی کند.

Total را به‌روزرسانی کند.

در صورت نامعتبر بودن Quantity، سیستم نباید مقدار نامعتبر را ثبت کند.

## 8. حذف محصول

Customer باید بتواند یک Cart Item را حذف کند.

حذف Item نباید باعث حذف Orderهای قبلی شود.

Cart و Order باید از نظر Lifecycle مستقل باشند.

## 9. محاسبات Cart

سیستم باید حداقل موارد زیر را محاسبه کند:

Subtotal

مجموع ارزش محصولات قبل از هزینه‌ها و تخفیف‌ها.

Discount

مجموع تخفیف‌های معتبر.

Shipping / Delivery Cost



هزینه ارسال در صورت مشخص بودن در این مرحله.

Additional Charges

هزینه‌های قابل اعمال در صورت وجود.

Grand Total

مبلغ نهایی قابل پرداخت.

## 10. Price Snapshot

در زمان تبدیل Cart به Order، قیمت محصول باید به صورت Snapshot در Order ثبت شود.

سیستم نباید برای Order نهایی صرفاً به قیمت فعلی Product متکی باشد.

اطلاعات ثبت شده باید حداقل شامل:

Product ID

Product Name Snapshot

Variant ID

Unit Price

Quantity

Discount

Line Total

باشد.

## 11. Order Creation

پس از تأیید نهایی اطلاعات توسط Customer، سیستم باید بتواند Order ایجاد کند.

Order باید دارای:

- Order ID
- Order Number
- Customer ID
- Order Items
- Pricing Information
- Delivery Information
- Payment Status
- Order Status
- Created At
- Updated At

باشد.

## Order Item .12

هر محصول موجود در Order باید به عنوان یک Order Item مستقل ثبت شود.

Order Item حداقل شامل:

Product ID

Variant ID در صورت وجود

Seller ID در صورت نیاز

Product Name Snapshot

Quantity

Unit Price

Discount

Total Price

است.

## Order Number .13

هر Order باید دارای شناسه داخلی یکتا و یک Order Number قابل نمایش برای Customer باشد.

Order Number نباید به نحوی تولید شود که احتمال Duplicate شدن آن وجود داشته باشد.

## Order Status .14

وضعیت‌های MVP باید حداقل قابلیت پشتیبانی از Lifecycle زیر را داشته باشند:

Pending

سفارش ایجاد شده ولی هنوز در وضعیت نهایی پردازش قرار نگرفته است.

Payment Pending

پرداخت مورد نیاز انجام نشده است.

Paid

پرداخت موفق ثبت شده است.

Processing

سفارش وارد فرآیند پردازش شده است.

Preparing

سفارش در حال آماده‌سازی است.

Shipped

سفارش ارسال شده است.

Delivered

سفارش تحویل شده است.

Cancelled

سفارش لغو شده است.

Failed

پردازش سفارش با خطای غیرقابل ادامه مواجه شده است.

### 15. Order State Transition

تغییر Status باید کنترل‌شده باشد.

سیستم نباید اجازه دهد هر Status به‌صورت آزادانه به Status دیگر تبدیل شود.

مثلاً:

Pending → Payment Pending → Paid → Processing → Preparing → Shipped → Delivered

و مسیرهای مربوط به:

Cancellation / Failure / Refund

باید مطابق Business Rule‌های نهایی Commerce طراحی شوند.

### 16. Payment Integration

Cart & Order Management باید قابلیت اتصال به Payment Module را داشته باشد.

Order باید Payment Status مستقل داشته باشد.

نمونه وضعیت‌ها:

- Unpaid
- Pending
- Paid
- Failed
- Refunded
- Partially Refunded
- منطق کامل Payment در SRS اختصاصی Payment Module تعریف خواهد شد.

- این مازول صرفاً باید Interface مناسب برای ارتباط با Payment فراهم کند.

### **17. Cart و Payment**

Cart نباید به عنوان مدرک قطعی پرداخت تلقی شود.

پرداخت باید به Order معتبر متصل شود.

بنابراین:

Cart  $\neq$  Order  $\neq$  Payment

این سه Entity باید از نظر Domain و Lifecycle تفکیک شوند.

### **18. Order و Fulfillment**

Order باید قابلیت اتصال به Fulfillment داشته باشد.

اطلاعاتی مانند:

Fulfillment Status

Shipment

Delivery Method

Delivery Address

Pickup Point

نباید به شکل Hard-Coded در Cart پیاده سازی شوند.

### **19. Order History**

Customer باید بتواند لیست Orderهای خود را مشاهده کند.

حداقل اطلاعات:

Order Number

Date

Total

Status

Item Count

نمایش داده شود.

## 20. Order Detail

Customer باید بتواند جزئیات Order را مشاهده کند.

حداقل:

Order Number

تاریخ

محصولات

Quantity

Unit Price

Discount

Total

Payment Status

Order Status

Delivery Information

نمایش داده شود.

## 21. Empty Cart

در صورت خالی بودن Cart، سیستم باید Empty State مناسب نمایش دهد.

کاربر باید بتواند از Empty Cart به مسیر Browse/Search Products بازگردد.

## 22. Out of Stock

اگر محصول موجود نباشد:

سیستم نباید اجازه ثبت Quantity نامعتبر بدهد.

وضعیت موجودی باید به کاربر نمایش داده شود.

Checkout نباید با Item نامعتبر انجام شود.

## 23. Price Change

اگر قیمت Product بین زمان Add to Cart و زمان ایجاد Order تغییر کرده باشد، سیستم باید Price Validation انجام دهد.

رفتار دقیق Price Change باید مطابق Pricing Policy ویرایش تعیین شود.

تا زمان تصویب Business Rule نهایی، Developer نباید رفتار خاصی را به صورت مستقل قطعی کند.

## **Inventory Validation .24**

قبل از نهایی شدن Order باید موجودی دوباره بررسی شود.

Cart نباید به تنهایی باعث رزرو دائمی Inventory شود، مگر اینکه Inventory Policy به صورت جداگانه چنین رفتاری را تعریف کند.

## **Multi-Seller Order .25**

از آنجا که ویرایش Marketplace است، معماری باید امکان وجود محصولات چند Seller در یک Cart را در آینده در نظر بگیرد.

در MVP، اگر Multi-Seller Order فعال باشد، Order باید قابلیت تفکیک Seller-level information را داشته باشد.

این طراحی نباید مانع توسعه:

Seller Order

Fulfillment Order

Settlement

در آینده شود.

## **Group Buying با ارتباط .26**

Group Buying باید به صورت مستقل از Cart/Order Domain طراحی شود، اما بتواند از زنجیره استاندارد Order استفاده کند.

در معماری آینده:

Group Order → Child Order → Payment → Fulfillment

باید امکان پذیر باشد.

Cart & Order Management نباید Group Buying را به یک سیستم کاملاً موازی تبدیل کند.

## **Business Rules .27**

BR-01

Cart متعلق به Customer یا Session معتبر است.

BR-02

هر Cart می تواند چند Cart Item داشته باشد.

BR-03

هر Cart Item باید به Product معتبر متصل باشد.

BR-04

Quantity باید مطابق محدودیت Product/Seller باشد.

BR-05

Order پس از ایجاد دارای Snapshot اطلاعات تجاری موردنیاز است.

BR-06

تغییر Product Price نباید Order قبلی را تغییر دهد.

BR-07

Payment باید به Order متصل باشد.

BR-08

Order Status و Payment Status باید از یکدیگر مستقل باشند.

BR-09

تغییر وضعیت Order باید از طریق State Transition معتبر انجام شود.

BR-10

Order پس از نهایی شدن نباید به صورت مستقیم از طریق Cart تغییر کند.

## **Functional Requirements .28**

FR-01 — Create Cart

سیستم باید امکان ایجاد یا بازیابی Cart فعال Customer را فراهم کند.

FR-02 — Add Item

سیستم باید امکان افزودن Product معتبر به Cart را فراهم کند.

FR-03 — Update Quantity

سیستم باید امکان تغییر Quantity را فراهم کند.

FR-04 — Remove Item

سیستم باید امکان حذف Cart Item را فراهم کند.

FR-05 — Validate Inventory

سیستم باید موجودی را اعتبارسنجی کند.

FR-06 — Calculate Cart

سیستم باید مبلغ Cart را محاسبه کند.

FR-07 — Validate Pricing

سیستم باید قیمت را قبل از ایجاد Order اعتبارسنجی کند.

#### FR-08 — Create Order

سیستم باید بتواند Cart معتبر را به Order تبدیل کند.

#### FR-09 — Generate Order Number

سیستم باید Order Number یکتا ایجاد کند.

#### FR-10 — Store Order Snapshot

سیستم باید اطلاعات تجاری Order را در زمان ایجاد Snapshot کند.

#### FR-11 — View Orders

Customer باید بتواند Orderهای خود را مشاهده کند.

#### FR-12 — View Order Detail

Customer باید بتواند جزئیات Order را مشاهده کند.

#### FR-13 — Track Order Status

Customer باید بتواند وضعیت Order را مشاهده کند.

#### FR-14 — Payment Integration

Order باید قابلیت اتصال به Payment Module را داشته باشد.

#### FR-15 — Fulfillment Integration

Order باید قابلیت اتصال به Fulfillment Module را داشته باشد.

#### FR-16 — Error Handling

سیستم باید خطاهای Cart و Order را مدیریت و پیام مناسب ارائه کند.

#### FR-17 — Authorization

کاربر فقط باید بتواند Cart و Orderهای مجاز خود را مشاهده یا مدیریت کند.

### API Requirements .29

APIهای موردنیاز باید حداقلیتهای زیر را پوشش دهند:

- Cart
- Create / Get Active Cart
- Get Cart
- Add Cart Item
- Update Cart Item
- Remove Cart Item
- Clear Cart



- Validate Cart
- Calculate Cart
- Order
- Create Order
- Get Order
- List Customer Orders
- Cancel Order در صورت مجاز بودن
- Get Order Status

نامگذاری نهایی Endpoint ها توسط CTO/Backend Developer تعیین می‌شود.

### 30. Authentication & Authorization

دسترسی به Cart و Order باید بر اساس Authentication و Authorization انجام شود.

Customer نباید بتواند:

Order متعلق به Customer دیگر را مشاهده کند.

Order دیگری را تغییر دهد.

Payment دیگری را مشاهده کند.

اطلاعات حساس Order دیگران را دریافت کند.

### 31. Security Requirements

سیستم باید حداقل موارد زیر را رعایت کند:

Authentication معتبر

Authorization در سطح Resource

Validation ورودی‌ها

جلوگیری از IDOR

Rate Limiting در Endpoint های حساس

عدم اعتماد به قیمت ارسال شده از Client

عدم اعتماد به Quantity ارسال شده بدون Validation

ثبت Audit Log برای تغییرات حساس

جلوگیری از دستکاری Order

Secure Error Handling

قیمت نهایی Order باید توسط Backend محاسبه یا اعتبارسنجی شود و Client نباید مرجع قابل اعتماد قیمت باشد.

### 32. Concurrency

در شرایطی که چند کاربر همزمان یک محصول محدود را خریداری می‌کنند، Backend باید از ایجاد Overselling جلوگیری کند. جزئیات Inventory Reservation / Transaction / Locking مطابق معماری Inventory Module توسط CTO تعیین خواهد شد.

### 33. Transaction Integrity

ایجاد Order باید تا حد امکان Atomic باشد.

در صورت Failure در مراحل حساس:

Order نباید در وضعیت ناقص و نامعتبر باقی بماند.

Inventory نباید بدون کنترل تغییر کند.

Payment Reference نباید به Order اشتباه متصل شود.

### 34. Logging

موارد مهم باید Log شوند:

- Cart Creation
- Add Item
- Update Quantity
- Remove Item
- Order Creation
- Order Status Change
- Payment Status Change
- Cancellation
- System Error

اطلاعات حساس پرداخت نباید در Log ذخیره شود.

### 35. Notification

در MVP حداقل Notification های مرتبط با Order باید قابلیت اتصال داشته باشند.

نمونه:

- Order Created
- Payment Successful
- Payment Failed
- Order Processing

- Order Shipped
- Order Delivered
- Order Cancelled

کانال Notification می‌تواند در Notification Module مدیریت شود.

### 36. Performance Requirements

سیستم باید برای استفاده عادی Marketplace با پاسخدهی مناسب طراحی شود. محدودیت‌های دقیق Performance توسط CTO بر اساس معماری و Infrastructure نهایی تعیین می‌شود. Cart و Order Endpointها نباید Queryهای غیرضروری و سنگین داشته باشند. Database Indexing مناسب برای:

- Customer ID
- Cart ID
- Order ID
- Order Number
- Status
- Created At

باید در نظر گرفته شود.

### 37. Database Requirements

مدل داده باید حداقل قابلیت پشتیبانی از Entityهای زیر را داشته باشد:

- Cart
- CartItem
- Order
- OrderItem
- OrderStatus
- Payment Reference
- Fulfillment Reference

نام Entityها و Schema نهایی توسط Backend Architect تعیین می‌شود.

### 38. Frontend / UI Requirements

صفحات MVP مرتبط با این SRS حداقل شامل:

- Cart
- Cart Page
- Empty Cart

- Cart Loading
- Cart Error
- Quantity Update
- Remove Item
- Price Summary
- Order
- Checkout / Order Confirmation
- Order Success
- Order Failed
- Order Detail
- Order History
- Order Status

تمام صفحات باید مطابق Figma تأییدشده پیادهسازی شوند.

### 39. Responsive Requirements

UI باید برای:

Desktop

Tablet

Mobile

Responsive باشد.

Mobile UX باید برای عملیات زیر بهینه باشد:

- مشاهده Cart
- تغییر Quantity
- حذف Item
- مشاهده مبلغ نهایی
- ایجاد Order
- مشاهده وضعیت Order

### 40. Error States

حداقل State های زیر باید طراحی و پیادهسازی شوند:

- Empty Cart
- Product Unavailable
- Out of Stock
- Invalid Quantity
- Price Changed
- Cart Validation Failed

- Order Creation Failed
- Payment Pending
- Payment Failed
- Unauthorized
- Session Expired
- Server Error
- Network Error

#### **Testing Requirements .41**

حداقل موارد زیر باید تست شوند:

- Unit Test
- Cart Calculation
- Quantity Validation
- Price Validation
- Order Number Generation
- Order Status Transition
- Integration Test
- Product → Cart
- Cart → Order
- Order → Payment
- Order → Fulfillment
- API Test
- تمام Endpoint های اصلی باید تست شوند.
- Security Test
- Unauthorized Access
- IDOR
- Invalid Input
- Price Manipulation
- Quantity Manipulation
- Edge Cases
- موجودی صفر
- موجودی کمتر از Quantity
- تغییر قیمت
- Cart خالی
- Duplicate Request
- Concurrent Order
- Payment Failure
- Order Creation Failure

#### **Acceptance Criteria .42**

ماژول زمانی قابل قبول است که:

- Customer بتواند Product را به Cart اضافه کند.
- Customer بتواند Quantity را تغییر دهد.
- Customer بتواند Item را حذف کند.
- مبلغ Cart به درستی محاسبه شود.
- موجودی اعتبارسنجی شود.
- قیمت قبل از Order Validation شود.
- Order با اطلاعات صحیح ایجاد شود.
- Order Number یکتا باشد.
- Order Item ها صحیح ثبت شوند.
- Payment به Order صحیح متصل شود.
- Customer بتواند Order های خود را مشاهده کند.
- Customer بتواند جزئیات Order را مشاهده کند.
- Order Status صحیح نمایش داده شود.
- دسترسی غیرمجاز به Order ها مسدود شود.
- Error State ها مدیریت شوند.
- Test های مورد نیاز اجرا و موفق شوند.
- Integration های لازم تکمیل شوند.
- Code Review انجام شود.
- Deployment انجام شود.
- Product Owner بتواند Flow نهایی را طبق Figma تست و تأیید کند.

#### 43. Definition of Done

این SRS زمانی Done محسوب می شود که:

Development → Testing → Integration → Code Review → Security Review → Deployment → Product Review → Acceptance

به صورت کامل انجام شده باشد.

صرفاً نوشته شدن Source Code به عنوان تحویل ماژول محسوب نمی شود.

#### 44. وابستگی ها

این ماژول به صورت مستقیم یا غیرمستقیم با موارد زیر ارتباط دارد:

- Authentication & User Management
- Product Catalog
- Seller Management
- Inventory
- Pricing
- Payment

- Fulfillment
- Shipment
- Notification
- Admin & Management
- Group Buying در مراحل بعدی

## SRS-08 — Payment & Transaction Management

سند نیازمندی‌های نرم‌افزاری

### 1. مشخصات سند

نام پروژه: Virashops

نام ماژول: Payment & Transaction Management

کد SRS: SRS-08

نسخه: 1.0

وضعیت: MVP

نوع سند: Software Requirements Specification

مخاطبان: Product Owner، CTO، Backend، Frontend، QA، UI/UX

اولویت: Critical / Must Have

### 2. هدف سند

هدف این ماژول طراحی و پیاده‌سازی لایه مدیریت پرداخت و تراکنش‌های مالی ویراشاپس است.

این ماژول باید امکان اتصال امن و قابل اعتماد بین:

Order → Payment → Transaction → Settlement

را فراهم کند.

Payment باید از Order مستقل باشد اما به آن Reference داشته باشد و وضعیت پرداخت نباید صرفاً بر اساس وضعیت Order تعیین شود.

### 3. محدوده MVP

موارد زیر در Scope نسخه MVP قرار دارند:

ایجاد Payment

ایجاد Payment Intent

اتصال Order به Payment  
اتصال Customer به Payment  
اتصال Transaction به Payment  
اتصال Payment Gateway به  
Redirect / Callback  
Verification  
ثبت تراکنش موفق  
ثبت تراکنش ناموفق  
مدیریت Payment Pending  
مدیریت Duplicate Payment  
ثبت Transaction Reference  
ثبت Gateway Reference  
مدیریت خطاهای Gateway  
Payment Status  
Transaction Status  
Order Payment Status  
Refund در سطح موردنیاز MVP  
ثبت Audit اطلاعات حساس  
Notification پایه پرداخت  
گزارش تراکنش‌های موردنیاز Admin  
Validation و Security  
Logging  
Testing  
Deployment

#### 4. خارج از Scope MVP

موارد زیر در مراحل آینده قابل توسعه هستند:

Wallet کامل

اعتبار خرید



BNPL

Credit Scoring

پرداخت اقساطی پیشرفته

پرداخت چکی پیشرفته

Split Payment پیشرفته

Group Payment پیشرفته

Multi-Gateway Intelligent Routing

Payment Optimization با AI

Fraud Detection پیشرفته

Financial AI

Automated Reconciliation پیشرفته

Currency Management چندگانه

International Payment

## 5. Entity های اصلی

معماری باید حداقل قابلیت پشتیبانی از Entity های زیر را داشته باشد:

Payment

Transaction

PaymentAttempt

PaymentGateway

Refund

PaymentStatus

TransactionStatus

PaymentMethod

GatewayReference

نامگذاری نهایی توسط CTO تعیین می شود.

## 6. Payment Lifecycle

چرخه اصلی پرداخت:

Created → Pending → Redirected → Processing → Verified → Paid

و مسیرهای خطا:

Pending → Failed

Processing → Failed

Paid → Refunded

و در صورت نیاز:

Paid → Partially Refunded

## **Payment Creation .7**

پس از ایجاد Order معتبر، سیستم باید بتواند Payment ایجاد کند.

Payment حداقل شامل:

Payment ID

Order ID

Customer ID

Amount

Currency

Payment Method

Status

Created At

Updated At

باشد.

## **Payment Amount .8**

مبلغ Payment باید توسط Backend محاسبه یا از Order معتبر دریافت شود.

Client نباید بتواند مبلغ نهایی پرداخت را تعیین کند.

Backend باید قبل از ایجاد Payment بررسی کند:

Order معتبر است.

Order متعلق به Customer صحیح است.

Order قابل پرداخت است.

مبلغ Order معتبر است.

Payment قبلی موفق برای همان Order وجود ندارد.

## **9. Payment Gateway**

سیستم باید قابلیت اتصال به Gateway را داشته باشد.

Gateway Adapter باید تا حد امکان از Business Logic مستقل باشد.

ساختار پیشنهادی:

Payment Service → Gateway Adapter → External Gateway

تا در آینده امکان اضافه کردن Gateway های دیگر بدون تغییر گسترده Payment Domain وجود داشته باشد.

## **10. Payment Attempt**

هر تلاش برای پرداخت باید در صورت نیاز به صورت مستقل ثبت شود.

مثلاً:

Payment:

PAY-10001

Attempts:

Attempt 01 → Failed

Attempt 02 → Failed

Attempt 03 → Successful

این ساختار برای Audit، Debug و جلوگیری از ابهام در تراکنش ها ضروری است.

## **11. Transaction**

Transaction باید اطلاعات مربوط به عملیات مالی را ثبت کند.

حداقل:

Transaction ID

Payment ID

Order ID

Amount

Gateway

Gateway Reference

Status

Created At

Verified At

را نگهداری کند.

## **Gateway Reference .12**

شناسه‌ای که Gateway ارائه می‌کند باید در سیستم ثبت شود.

Gateway Reference نباید تنها معیار شناسایی داخلی Transaction باشد.

سیستم باید دارای Internal Transaction ID مستقل باشد.

## **Callback .13**

پس از بازگشت کاربر از Gateway، سیستم باید Callback را دریافت کند.

Callback نباید به‌تنهایی به‌عنوان Payment Success پذیرفته شود.

پس از Callback باید Verification انجام شود.

## **Payment Verification .14**

Backend باید تراکنش را با Gateway Verify کند.

پس از Verification موفق:

Transaction → Success

Payment → Paid

Order Payment Status → Paid

در صورت Verification ناموفق:

Transaction → Failed

Payment → Failed

## **Idempotency .15**

Payment System باید در برابر درخواست‌های تکراری مقاوم باشد.

اگر Callback یا Request چند بار دریافت شود، نباید:

چند Payment ایجاد شود.

چند Transaction موفق ثبت شود.

مبلغ چند بار به Order اختصاص داده شود.

#### **16. Duplicate Payment**

سیستم باید بتواند Payment موفق قبلی را تشخیص دهد.

اگر Order قبلاً Paid شده باشد، ایجاد Payment جدید نباید بدون Business Rule معتبر انجام شود.

#### **17. Payment Status**

وضعیت‌های MVP:

Created

Pending

Processing

Paid

Failed

Cancelled

Refunded

Partially Refunded

#### **18. Transaction Status**

Transaction Status حداقل:

Created

Pending

Processing

Successful

Failed

Reversed

Refunded

را پشتیبانی کند.

## **Order Payment Status .19**

Payment Status مربوط به Order باید مستقل از Order Status باشد.

مثلاً:

Order = Processing

اما:

Payment = Paid

یا:

Order = Pending

و:

Payment = Pending

## **Refund .20**

در MVP باید معماری Payment امکان Refund را در صورت نیاز Business داشته باشد.

Refund باید دارای Reference مستقل باشد.

اطلاعات:

Refund ID

Payment ID

Transaction ID

Amount

Reason

Status

Created At

ثبت شود.

## **Partial Refund .21**

در صورت نیاز MVP، سیستم باید امکان Partial Refund را در Architecture پیش‌بینی کند.

مثلاً:

Payment:

1,000,000 تومان

Refund:

300,000 تومان

Payment نباید به صورت کامل به Refunded تغییر کند مگر اینکه کل مبلغ Refund شده باشد.

## 22. Payment Failure

در صورت Failure:

وضعیت Payment ثبت شود.

Transaction ثبت شود.

Error Code در صورت نیاز ذخیره شود.

اطلاعات حساس Gateway ذخیره نشود.

Customer پیام مناسب دریافت کند.

امکان Retry در صورت مجاز بودن فراهم باشد.

## 23. Retry

Customer در صورت Payment Failure باید بتواند در شرایط مجاز دوباره پرداخت کند.

Retry نباید باعث ایجاد وضعیت‌های متناقض در Order شود.

## 24. Security

Payment Module باید الزامات امنیتی بالایی داشته باشد.

حداقل:

- HTTPS
- Authentication
- Authorization
- Input Validation
- Rate Limiting
- Idempotency
- Secure Callback Handling
- Gateway Verification
- جلوگیری از Replay Attack
- جلوگیری از Payment Manipulation
- عدم اعتماد به Client Amount

- Audit Logging

## 25. اطلاعات حساس

اطلاعات حساس کارت بانکی نباید در Database یا Log های سیستم ذخیره شود، مگر در حدودی که قانون و Gateway/Payment Provider صراحتاً اجازه دهد.

سیستم باید تا حد امکان از Tokenization و Gateway-hosted Payment استفاده کند.

## 26. API Requirements

API های اصلی باید قابلیت های زیر را فراهم کنند:

- Payment
- Create Payment
- Get Payment
- Initiate Payment
- Verify Payment
- Retry Payment
- Cancel Payment در صورت مجاز بودن
- Transaction
- Get Transaction
- Verify Transaction
- Refund
- Create Refund
- Get Refund

نام Endpoint ها و قرارداد API توسط CTO نهایی می شود.

## 27. Authentication & Authorization

Customer فقط باید بتواند Payment های مجاز مربوط به Order های خودش را مشاهده کند.

Admin فقط در صورت داشتن Permission مناسب باید بتواند اطلاعات تراکنش ها را مشاهده کند.

هیچ Customer نباید بتواند Payment یا Transaction مربوط به Customer دیگری را مشاهده کند.

## 28. Database Requirements

Database باید امکان ذخیره موارد زیر را داشته باشد:

Payment

id



order\_id  
customer\_id  
amount  
currency  
method  
status  
created\_at  
updated\_at  
Transaction  
id  
payment\_id  
gateway  
gateway\_reference  
amount  
status  
created\_at  
verified\_at  
Refund  
id  
payment\_id  
amount  
status  
reason  
created\_at

## Transaction Integrity .29

عملیات مالی باید Transactionally Safe باشند.

در صورت Failure نباید وضعیت سیستم به شکل زیر باقی بماند:

Gateway = Success

اما:

Internal Payment = Failed

مکانیزم Reconciliation و Verification باید برای جلوگیری از چنین اختلاف‌هایی پیش‌بینی شود.

### 30. Reconciliation

در MVP حداقل قابلیت بررسی اختلاف بین:

Internal Transaction

و

Gateway Transaction

باید وجود داشته باشد.

در صورت مشاهده اختلاف، Transaction باید قابل بررسی توسط Admin باشد.

### 31. Logging

Log‌های مرتبط با:

- Payment Creation
- Payment Attempt
- Gateway Request
- Gateway Response
- Verification
- Payment Failure
- Refund
- Status Change

باید ثبت شوند.

اطلاعات حساس نباید در Log قرار گیرد.

### 32. Notification

سیستم باید قابلیت ارسال Notification برای:

Payment Created

Payment Successful

Payment Failed

Refund Successful

Refund Failed

را فراهم کند.

Notification Service مسئول کانال ارسال خواهد بود.

### Admin Requirements .33

Admin باید بتواند در سطح MVP:

- Payment ها را مشاهده کند.
- Transaction ها را مشاهده کند.
- Status را مشاهده کند.
- Gateway Reference را مشاهده کند.
- Order مرتبط را مشاهده کند.
- Customer مرتبط را مشاهده کند.
- Refund های ثبت شده را مشاهده کند.
- Transaction های مشکوک یا ناموفق را بررسی کند.

### Integration .34

Payment Module باید با موارد زیر Integration داشته باشد:

- Order Management
- برای دریافت Order و Amount.
- User Management
- برای شناسایی Customer.
- Gateway
- برای عملیات پرداخت.
- Notification
- برای اطلاع رسانی.
- Admin
- برای Monitoring و بررسی.
- Settlement
- در مراحل بعدی برای تسویه با Seller.

### Seller Settlement .35

Payment و Settlement نباید یک Entity یا یک فرآیند واحد در نظر گرفته شوند.

زنجیره پیشنهادی:

Customer Payment → Platform Transaction → Order → Seller Settlement

Settlement با Seller در SRS/Domain جداگانه مدیریت خواهد شد.

### 36. Group Buying Integration

در آینده Payment باید بتواند از Group Order پشتیبانی کند.

ساختار:

Master Group Order

→

Child Order

→

Payment

→

Transaction

هر عضو باید بتواند Payment مستقل داشته باشد.

### 37. B2B / Wholesale Integration

Payment Architecture باید در آینده قابلیت پشتیبانی از:

- نقدی
- کارت
- چکی
- اعتباری
- اقساطی
- پرداخت مرحله‌ای
- Split Payment

را داشته باشد.

این موارد در MVP به صورت کامل پیاده‌سازی نمی‌شوند مگر اینکه در Scope رسمی اضافه شده باشند.

### 38. Error Handling

خطاهای اصلی:

- Gateway Timeout
- Gateway Failure
- Invalid Payment
- Invalid Amount
- Duplicate Payment
- Verification Failure

- Network Failure
- Order Not Payable
- Payment Expired
- Unauthorized Payment
- Refund Failure

باید مدیریت شوند.

### 39. Performance

Payment API ها باید با Latency پایین و قابل پیش‌بینی طراحی شوند.  
در عملیات Gateway Timeout، سیستم نباید بدون Verification وضعیت Payment را Failed اعلام کند.  
Transaction باید امکان بررسی مجدد داشته باشد.

### 40. Testing

- Unit Test
- Payment Amount
- Status Transition
- Idempotency
- Refund Calculation
- Partial Refund
- Validation
- Integration Test
- Order → Payment
- Payment → Gateway
- Gateway → Verification
- Payment → Notification
- Security Test
- Amount Manipulation
- Callback Manipulation
- Unauthorized Access
- Replay Attack
- Duplicate Request
- Failure Test
- Gateway Timeout
- Gateway Failure
- Callback Failure
- Verification Failure
- Network Failure

#### 41. Acceptance Criteria

ماژول زمانی قابل قبول است که:

- Payment برای Order معتبر ایجاد شود.
- Amount از Backend معتبر دریافت شود.
- Payment به Order صحیح متصل شود.
- Gateway Integration انجام شده باشد.
- Callback دریافت شود.
- Verification انجام شود.
- Payment موفق به درستی ثبت شود.
- Payment ناموفق به درستی ثبت شود.
- Duplicate Payment کنترل شود.
- Idempotency رعایت شود.
- Transaction صحیح ثبت شود.
- Gateway Reference ثبت شود.
- Customer فقط Payment های مجاز خود را ببیند.
- Refund در Scope تعریف شده قابل انجام باشد.
- Error Handling تکمیل باشد.
- Logging انجام شود.
- تست های لازم اجرا شوند.
- Security Review انجام شود.
- Integration با Order تکمیل شود.
- Deployment انجام شود.
- Product Owner Flow پرداخت را تأیید کند.

#### 42. Definition of Done

Payment Module زمانی Done محسوب می شود که:

Development → Integration → Testing → Security Review → Code Review → Deployment → Product Review → Acceptance

به طور کامل انجام شده باشد.

صرفاً اتصال به Payment Gateway به عنوان تکمیل ماژول محسوب نمی شود.

#### 43. خروجی مورد انتظار Developer

Developer باید تحویل دهد:

- Source Code
- Payment APIs
- Gateway Integration
- Database Migration

- Unit Tests
- Integration Tests
- Security Tests
- Documentation
- Error Handling
- Logging
- Deployment
- Technical Notes
- Known Limitations

#### 44. الزامات معماری آینده

Payment Module باید:

Modular

Maintainable

Secure

Extensible

Testable

Gateway-independent

API-driven

باشد.

Business Logic پرداخت نباید مستقیماً به UI یا Framework خاص Frontend وابسته شود.

معماری باید امکان توسعه آینده به:

Payment → Credit → Installment → Check → Wallet → Split Payment → Financial Services

را فراهم کند.

#### 45. موارد نیازمند تأیید CTO / Product Owner

موارد زیر قبل از Implementation نهایی باید مشخص شوند:

Payment Gateway مورد استفاده

Payment Expiration

Retry Policy

Refund Policy

Partial Refund

Reconciliation Strategy

Settlement Architecture

Multi-Gateway Strategy

Currency

مالیات و کارمزدهای پرداخت

Group Payment

B2B Payment

Check Payment

Installment Payment

آینده Roadmap در Wallet / Credit

هیچ Business Rule مالی جدیدی بدون تأیید Product Owner و CTO نباید در Implementation نهایی وارد شود.

## **Traceability .46**

تمام Requirements این SRS باید قابلیت Trace شدن به:

Feature → User Flow → Figma → API → Backend Logic → Payment Gateway → Test Case → Acceptance Criteria

را داشته باشند.

## **SRS-09 — Admin & Management Panel**

سند نیازمندی‌های نرم‌افزاری

### **1. مشخصات سند**

نام پروژه: Virashops

نام ماژول: Admin & Management Panel

کد SRS: SRS-01

نسخه: 1.0

وضعیت: MVP



نوع سند: Software Requirements Specification

مخاطبان: UI/UX ،QA ،Frontend ،Backend ،CTO ،Product Owner

اولویت: Critical / Must Have

## 2. هدف سند

هدف این ماژول ایجاد پنل مرکزی مدیریت ویرایش برای مدیریت و نظارت بر موجودیت‌ها و فرآیندهای اصلی سیستم در نسخه MVP است.

پنل مدیریت باید به‌عنوان لایه مدیریتی سیستم عمل کرده و امکان کنترل و مشاهده بخش‌های اصلی زیر را فراهم کند:

کاربران

فروشنده‌گان

محصولات

سفارش‌ها

دسته‌بندی‌ها

محتوای موردنیاز MVP

تنظیمات سیستم

## 3. محدوده MVP

ماژول Admin & Management Panel در MVP شامل بخش‌های زیر است:

Dashboard

User Management

Seller Management

Product Management

Order Management

Category Management

MVP Content Management

System Settings

Authentication

Authorization / RBAC

Filter و Search مدیریتی

Pagination

Status Management

Basic Reporting / Overview

Audit / Activity Logging در سطح مورد نیاز MVP

#### **.4 Admin Authentication**

ورود به پنل مدیریت باید از طریق Authentication امن انجام شود.

حداقل موارد:

Login

Logout

Session Management

Password Management

Authentication Validation

Access Control

پنل مدیریت نباید برای کاربران عادی قابل دسترسی باشد.

#### **.5 RBAC و Authorization**

سیستم باید امکان تعریف Role و Permission را داشته باشد.

نمونه Role ها:

Super Admin

Admin

Content Manager

Seller Manager

Order Manager

Role ها و Permission های نهایی توسط CTO و Product Owner تعیین می شوند.

هر Admin فقط باید به بخش هایی دسترسی داشته باشد که Permission لازم برای آنها را دارد.

#### **.6 Admin Dashboard**

Dashboard باید یک نمای کلی از وضعیت سیستم ارائه کند.

اطلاعات مورد نیاز MVP می تواند شامل موارد زیر باشد:

- تعداد کاربران
- تعداد فروشندگان
- تعداد محصولات
- تعداد سفارش ها
- سفارش های جدید
- سفارش های در انتظار بررسی
- وضعیت فروشندگان
- وضعیت محصولات
- وضعیت سفارش ها

Dashboard در MVP نیاز به Analytics پیشرفته ندارد.

## 7. User Management

Admin باید بتواند کاربران را مدیریت کند.

قابلیت ها:

مشاهده کاربران

Search

Filter

مشاهده اطلاعات کاربر

مشاهده وضعیت حساب

فعال / غیر فعال کردن کاربر

مشاهده سابقه سفارش ها در سطح مجاز

مشاهده اطلاعات پایه حساب

اطلاعات حساس کاربر باید مطابق Permission و Security Policy نمایش داده شود.

## 8. User Status

حداقل وضعیت های کاربر:

- Active
- Inactive
- Suspended
- Pending در صورت نیاز

تغییر Status باید توسط Admin مجاز انجام شود.

## **Seller Management .9**

Admin باید بتواند فروشندگان را مدیریت کند.

قابلیت‌ها:

مشاهده Sellerها

Search

Filter

مشاهده اطلاعات Seller

بررسی وضعیت Seller

فعال / غیرفعال کردن Seller

مشاهده محصولات Seller

مشاهده سفارش‌های مرتبط

مشاهده اطلاعات پایه فروشنده

## **Seller Status .10**

وضعیت‌های پیشنهادی:

Pending

Active

Inactive

Suspended

Rejected

منطق نهایی Status توسط Product Owner و CTO مشخص می‌شود.

## **Product Management .11**

Admin باید بتواند محصولات Marketplace را مدیریت کند.

قابلیت‌ها:

مشاهده محصولات

Search

Filter

مشاهده جزئیات محصول

ایجاد محصول در صورت نیاز

ویرایش محصول

فعال / غیرفعال کردن محصول

مدیریت وضعیت انتشار

اتصال محصول به Category

مشاهده Seller مرتبط

## **Product Status .12**

حداقل:

Draft

Active

Inactive

Pending Review

Rejected

## **Category Management .13**

Admin باید بتواند ساختار Category را مدیریت کند.

قابلیت‌ها:

ایجاد Category

ویرایش Category

حذف Category در شرایط مجاز

فعال / غیرفعال کردن Category

مدیریت Parent / Child Category

مرتب‌سازی

مشاهده محصولات مرتبط

## **Category Hierarchy .14**

سیستم باید امکان ساختار Hierarchical Category را داشته باشد.

مثلاً:

Home Appliances

Refrigerators →

Washing Machines →

Television →

ساختار نهایی براساس Catalog Architecture ویرایش تعیین می‌شود.

## Order Management .15

Admin باید بتواند سفارش‌های سیستم را مدیریت و مشاهده کند.

قابلیت‌ها:

مشاهده Orders

Search

Filter

مشاهده Order Details

مشاهده Customer

مشاهده Seller

مشاهده Products

مشاهده Payment Status

مشاهده Fulfillment Status

مشاهده Order Status

تغییر Status در صورت مجاز بودن

## Order Status .16

Admin باید بتواند وضعیت سفارش را مشاهده کند.

نمونه Statusها:

Pending

Confirmed

Processing

Ready for Shipment

Shipped

Delivered

Cancelled

Refunded

Status های نهایی با Order Management SRS هماهنگ خواهند شد.

## 17. Content Management

پنل Admin باید امکان مدیریت محتوای ضروری MVP را داشته باشد.

موارد احتمالی:

Banner

Homepage Content

Promotional Content

Basic Pages

FAQ

Terms

Privacy

سایر محتوای ضروری MVP

CMS باید تا حد امکان از Business Logic اصلی سیستم جدا باشد.

## 18. System Settings

Admin مجاز باید بتواند تنظیمات عمومی سیستم را مدیریت کند.

نمونه:

اطلاعات عمومی Platform

تنظیمات عمومی سفارش

تنظیمات Notification

تنظیمات مربوط به Marketplace

تنظیمات عمومی فروشنده

تنظیمات عمومی محصولات

تنظیمات حساس Infrastructure نباید مستقیماً از طریق UI در اختیار Admin عادی قرار گیرد.

## 19. Search

بخش‌های مدیریتی باید قابلیت Search داشته باشند.

حداقل:

User Search

Seller Search

Product Search

Order Search

Category Search

Search باید بر اساس شناسه‌ها و فیلدهای اصلی Entity انجام شود.

## 20. Filter

برای جلوگیری از نمایش حجم زیاد داده، Filter باید در List‌های اصلی وجود داشته باشد.

نمونه:

- Users
- Status
- Registration Date
- Sellers
- Status
- Category
- Products
- Status
- Seller
- Category
- Orders
- Status
- Payment Status
- Date
- Seller
- Customer

## 21. Pagination

List‌های مدیریتی باید Pagination داشته باشند.

Pagination باید Server-side باشد تا در حجم بالای داده، Performance سیستم حفظ شود.



## 22. Detail View

برای هر Entity اصلی باید صفحه Detail وجود داشته باشد.

حداقل:

User Detail

Seller Detail

Product Detail

Order Detail

Category Detail

اطلاعات Detail باید متناسب با Role و Permission نمایش داده شود.

## 23. CRUD Operations

CRUD کامل برای همه Entity ها الزاماً در MVP مورد نیاز نیست.

هر Entity باید براساس Business Requirement قابلیت های مورد نیاز خود را داشته باشد:

Create

Read

Update

Delete

Delete باید در موارد حساس ترجیحاً به صورت Soft Delete یا Deactivation پیاده سازی شود.

## 24. Audit & Activity Log

عملیات حساس Admin باید قابلیت ثبت Audit داشته باشند.

مانند:

- Login
- Logout
- User Status Change
- Seller Status Change
- Product Modification
- Order Status Change
- Category Modification
- System Setting Change
- Audit Log باید حداقل شامل:
- Actor

- Action
- Entity
- Entity ID
- Timestamp

باشد.

## 25. Security

Admin Panel باید الزامات امنیتی بالاتری نسبت به بخش عمومی سیستم داشته باشد.

حداقل:

Authentication

Authorization

RBAC

Input Validation

Secure Session

Rate Limiting

Protection Against Unauthorized Access

Audit Logging

Secure API

Server-side Permission Validation

Permission نباید صرفاً در Frontend کنترل شود.

تمام عملیات حساس باید در Backend نیز Authorization شوند.

## 26. API Architecture

Admin Panel باید از API های Backend استفاده کند.

Frontend نباید مستقیماً به Database متصل شود.

ساختار کلی:

Admin Frontend → API → Backend Services → Database

API ها باید دارای Authentication و Authorization باشند.

## 27. Database

Admin Panel Database مستقل و موازی با Core Business Database ایجاد نمی‌کند، مگر اینکه CTO براساس معماری نهایی تصمیم دیگری بگیرد.

Admin Panel باید از Domain‌های اصلی سیستم برای دریافت و مدیریت داده استفاده کند.

## 28. Frontend Requirements

Admin Panel باید:

- Responsive باشد.
- Component-based باشد.
- Maintainable باشد.
- Modular باشد.
- دارای Loading State باشد.
- دارای Empty State باشد.
- دارای Error State باشد.
- دارای Success Feedback باشد.

در صورت نیاز می‌تواند Desktop-first باشد، اما Responsive بودن بخش‌های اصلی باید حفظ شود.

## 29. UI States

صفحات اصلی باید حداقل State‌های زیر را داشته باشند:

Default

Loading

Empty

Error

Success

Disabled

Permission Denied

## 30. Error Handling

در صورت خطای API:

پیام مناسب نمایش داده شود.

اطلاعات فنی حساس نمایش داده نشود.

امکان Retry در موارد مناسب وجود داشته باشد.

Error در Backend Log شود.

### **Notification .31**

Admin باید در موارد مهم Notification دریافت کند.

مانند:

New Seller

New Order

Payment Issue

Product Pending Review

System Error در صورت نیاز

Notification های پیشرفته در مراحل بعدی قابل توسعه هستند.

### **Performance .32**

Admin Panel باید در حجم بالای داده نیز قابل استفاده باشد.

الزامات:

Server-side Pagination

Query Optimization

مناسب بودن Index های Database

جلوگیری از N+1 Query

Lazy Loading در موارد مناسب

Caching در موارد مورد نیاز

### **Testing .33**

Unit Test

برای Business Logic های مربوط به:

Permission

Status

Validation

CRUD

Integration Test

برای:

Admin → API

API → Backend

Authentication

Authorization

Database

Security Test

برای:

Unauthorized Access

Privilege Escalation

Role Bypass

API Access

Session Security

### 34. Acceptance Criteria

Admin & Management Panel زمانی قابل قبول است که:

- Admin بتواند Login کند.
- Authentication به درستی کار کند.
- Authorization در Backend اعمال شود.
- Role و Permission قابل کنترل باشد.
- Dashboard اصلی قابل مشاهده باشد.
- User Management فعال باشد.
- Seller Management فعال باشد.
- Product Management فعال باشد.
- Order Management فعال باشد.
- Category Management فعال باشد.
- MVP Content Management فعال باشد.
- System Settings مورد نیاز MVP فعال باشد.
- Search و Filter کار کند.
- Pagination کار کند.
- Detail View های اصلی وجود داشته باشد.
- Status Management صحیح باشد.
- Audit عملیات حساس ثبت شود.

- Error Handling تکمیل باشد.
- Responsive UI تکمیل باشد.
- تست‌های موردنیاز اجرا شوند.
- Security Review انجام شود.
- Deployment انجام شود.
- Product Owner Flow های اصلی را تأیید کند.

### 35. Definition of Done

ماژول زمانی Done محسوب می‌شود که:

Development → API Integration → Testing → Security Review → Code Review → Deployment →  
Product Review → Acceptance

به‌طور کامل انجام شده باشد.

صرفاً ساختن صفحات Admin به‌عنوان تکمیل ماژول محسوب نمی‌شود.

### 36. خروجی مورد انتظار Developer

Developer باید موارد زیر را تحویل دهد:

- Source Code
- Admin Frontend
- API Integration
- Backend Integration
- Database Changes در صورت نیاز
- Authentication
- Authorization
- RBAC
- Tests
- Documentation
- Error Handling
- Audit Logging
- Deployment
- Technical Notes
- Known Limitations

### 37. وابستگی‌ها

این ماژول با بخش‌های زیر ارتباط دارد:

User Management

Seller Management  
Product Catalog  
Order Management  
Payment Management  
Content Management  
Authentication  
Authorization  
Notification

### 38. الزامات معماری آینده

Admin Panel باید به صورت:

- Modular
- Maintainable
- Scalable
- Secure
- API-driven
- Role-based

طراحی شود.

Business Logic نباید مستقیماً به UI وابسته باشد.

ساختار باید امکان توسعه آینده برای:

Advanced Analytics  
Seller Management  
Financial Management  
Logistics Management  
AI Administration  
Fraud Management  
Recommendation Management  
Advertising Management  
Marketplace Management

را فراهم کند.

### 39. موارد نیازمند تأیید CTO / Product Owner

موارد زیر قبل از Implementation نهایی باید مشخص شوند:

Admin Frontend Framework

ساختار API

RBAC Model

Role های اولیه

Permission های اولیه

Audit Log Strategy

Dashboard Metrics

System Settings دقیق MVP

Content Management Scope

سطح دسترسی هر Role

Soft Delete Policy

Deployment Architecture

هر قابلیت خارج از Scope این SRS باید قبل از Implementation به عنوان Change Request ثبت و تأیید شود.

### 40. Traceability

تمام Requirements این SRS باید قابلیت Trace شدن به:

Feature → User Flow → Figma → API → Backend Logic → Database → Test Case → Acceptance Criteria

را داشته باشند.

### SRS-10 — Notification Management

Software Requirements Specification

#### 1. معرفی مازول

نام مازول: Notification Management

شناسه: SRS-10



وضعیت: MVP

نوع: Core Platform Module

ماژول Notification Management مسئول ایجاد، مدیریت، ارسال، نمایش و ثبت اعلان‌های سیستمی برای کاربران، فروشندگان و مدیران ویرایش است.

هدف این ماژول این است که رویدادهای مهم سیستم به شکل قابل اتکا و قابل پیگیری به کاربر مربوطه اطلاع‌رسانی شوند.

## 2. اهداف ماژول

ماژول باید امکان موارد زیر را فراهم کند:

- ایجاد Notification بر اساس رویدادهای سیستم
- ارسال اعلان به کاربر مناسب
- نمایش اعلان در پنل کاربر
- نمایش تعداد اعلان‌های خوانده‌نشده
- مدیریت وضعیت خوانده‌شدن اعلان
- نگهداری تاریخچه اعلان‌ها
- پشتیبانی از انواع مختلف Notification
- ایجاد زیرساخت قابل توسعه برای کانال‌های ارتباطی آینده

## 3. کاربران و نقش‌های مرتبط

Notification Management می‌تواند برای گروه‌های زیر اعلان ایجاد کند:

- Buyer / Customer
  - وضعیت سفارش
  - وضعیت پرداخت
  - تغییر وضعیت ارسال
  - پیام‌های مهم سیستم
  - اطلاع‌رسانی‌های مرتبط با خرید
- Seller
  - سفارش جدید
  - تغییر وضعیت سفارش
  - پرداخت یا تسویه
  - هشدار موجودی
  - پیام‌های مدیریتی
  - اطلاع‌رسانی‌های مرتبط با فروشگاه
- Admin / Management
  - رویدادهای مهم سیستم
  - هشدارهای مدیریتی
  - فعالیت‌های مهم کاربران و فروشندگان
  - خطاها یا رویدادهای نیازمند بررسی

#### Functional Requirements .4

##### Notification — FR-10-01 ایجاد

سیستم باید بتواند بر اساس رویدادهای مشخص شده، یک Notification ایجاد کند.

هر Notification حداقل باید شامل موارد زیر باشد:

Notification ID

Recipient User ID

Notification Type

Title

Message

Created At

Read Status

Related Entity

Related Entity ID

Priority

Status

##### FR-10-02 — Notification Types

سیستم باید از انواع مختلف اعلان پشتیبانی کند.

نمونه:

Order Notification

Payment Notification

Seller Notification

Product Notification

System Notification

Security Notification

Promotional Notification

ساختار باید به گونه ای باشد که در آینده امکان اضافه شدن Notification Type جدید بدون تغییر اساسی در معماری وجود داشته باشد.

### FR-10-03 — Notification Center

کاربر باید بتواند Notification Center خود را مشاهده کند.

Notification Center باید حداقل شامل موارد زیر باشد:

لیست اعلان‌ها

اعلان‌های خوانده‌نشده

اعلان‌های خوانده‌شده

زمان ایجاد

عنوان

خلاصه پیام

وضعیت اعلان

امکان ورود به بخش مرتبط

### FR-10-04 — Unread Counter

سیستم باید تعداد Notification های خوانده‌نشده کاربر را مشخص کند.

این مقدار می‌تواند در UI به شکل Badge یا Counter نمایش داده شود.

### FR-10-05 — Mark as Read

کاربر باید بتواند یک Notification را به عنوان خوانده‌شده علامت‌گذاری کند.

همچنین در صورت نیاز باید امکان:

Mark All as Read

وجود داشته باشد.

### FR-10-06 — Notification Detail

کاربر باید بتواند Notification را باز کرده و جزئیات کامل آن را مشاهده کند.

در صورت مرتبط بودن اعلان با یک Entity، سیستم باید امکان انتقال کاربر به بخش مربوطه را فراهم کند.

مثلاً:

Notification → Order → Order Detail

## FR-10-07 — Notification History

سیستم باید تاریخچه Notification های کاربر را نگهداری کند.

کاربر باید بتواند اعلان های قبلی خود را مشاهده کند، مگر اینکه طبق Policy سیستم حذف یا Archive شده باشند.

## FR-10-08 — Deep Link

Notification در صورت ارتباط با یک Entity باید بتواند کاربر را مستقیماً به صفحه مربوطه هدایت کند.

نمونه:

Order Notification → Order

Payment Notification → Payment

Product Notification → Product

Seller Notification → Seller Panel

## FR-10-09 — Priority

Notification ها باید امکان داشتن سطح اهمیت داشته باشند.

نمونه:

Low

Normal

High

Critical

اعلان های Critical باید قابلیت برخورد متفاوت در UI یا کانال ارسال را داشته باشند.

## FR-10-10 — Notification Status

سیستم باید وضعیت Notification را قابل مدیریت کند.

نمونه وضعیت ها:

Pending

Sent

Delivered

Read

Failed

در صورت پیاده‌سازی Channel های خارجی، وضعیت Delivery نیز باید قابل ثبت باشد.

## **Notification Channels .5**

### **In-App Notification 5.1**

کانال اصلی Notification در MVP، اعلان داخل سیستم است.

باید امکان نمایش Notification در:

**Web Application**

**User Panel**

**Seller Panel**

**Admin Panel**

وجود داشته باشد.

### **Email 5.2**

ساختار سیستم باید به گونه ای باشد که در صورت فعال شدن Email Notification، امکان ارسال اعلان از طریق Email وجود داشته باشد.

Email Template ها باید قابلیت توسعه داشته باشند.

### **SMS 5.3**

ساختار سیستم باید امکان اتصال آینده به SMS Provider را داشته باشد.

استفاده از SMS در MVP فقط در صورت قرار گرفتن در Scope نهایی پروژه انجام خواهد شد.

موارد احتمالی:

**OTP**

اطلاع رسانی های حساس

تغییر وضعیت های مهم

پیام های امنیتی

### **Push Notification 5.4**

Push Notification می تواند به عنوان قابلیت توسعه آینده در نظر گرفته شود.

معماری Notification نباید مانع اضافه شدن Push Notification در نسخه های بعدی شود.

## 6. Notification Preferences

در صورت قرار گرفتن در Scope MVP، کاربر باید بتواند تنظیمات دریافت Notification خود را مدیریت کند.

نمونه:

فعال/غیرفعال کردن Email

فعال/غیرفعال کردن SMS

فعال/غیرفعال کردن Push

تنظیم نوع Notification های دریافتی

Notification های ضروری و امنیتی ممکن است مستقل از تنظیمات اختیاری کاربر ارسال شوند.

## 7. Event-Based Notification

Notification Management باید قابلیت اتصال به رویدادهای سایر ماژول ها را داشته باشد.

نمونه:

Order Created

Notification →

Payment Successful

Notification →

Order Status Changed

Notification →

Seller Receives New Order

Notification →

بنابراین Notification Module نباید منطق Business Module های دیگر را در خود تکرار کند.

## 8. Integration Requirements

این ماژول باید قابلیت Integration با ماژول های زیر را داشته باشد:

- User Management
- Seller Management
- Product Catalog
- Cart & Order Management
- Payment

- Admin Management
- Authentication
- در آینده Logistics
- در آینده Marketing
- در آینده AI Services

## 9. Data Requirements

حداقل ساختار منطقی Notification باید امکان نگهداری اطلاعات زیر را داشته باشد:

- ID
- Recipient
- Type
- Title
- Message
- Channel
- Priority
- Status
- Read At
- Created At
- Updated At
- Related Entity Type
- Related Entity ID
- Metadata

Metadata باید امکان ذخیره اطلاعات تکمیلی Notification را بدون ایجاد وابستگی شدید به ساختار اصلی فراهم کند.

## 10. API Requirements

Backend باید API های موردنیاز برای موارد زیر را ارائه کند:

دریافت Notification های کاربر

دریافت تعداد Unread

دریافت Notification Detail

Mark as Read

Mark All as Read

دریافت Notification History

مدیریت Notification Preferences در صورت وجود در Scope

ایجاد Notification توسط سرویس های مجاز

## مدیریت وضعیت Notification

API ها باید Authentication و Authorization مناسب داشته باشند.

### 11. Security Requirements

کاربر فقط باید بتواند Notification های متعلق به خود را مشاهده کند.

دسترسی به Notification های سایر کاربران باید محدود به نقش های مجاز مدیریتی باشد.

اطلاعات حساس نباید در Notification به صورت غیرضروری نمایش داده شود.

Notification های مرتبط با عملیات امنیتی و احراز هویت باید با سطح دسترسی و کنترل مناسب مدیریت شوند.

### 12. Performance Requirements

Notification Center باید برای تعداد قابل توجهی Notification قابل استفاده باشد.

Pagination یا روش مناسب دیگری برای جلوگیری از دریافت حجم غیرضروری داده باید در نظر گرفته شود.

تعداد Unread باید با Query مناسب و بدون بار غیرضروری روی سیستم محاسبه شود.

ساختار Notification باید قابلیت توسعه به حجم بالاتر کاربران و اعلان ها را داشته باشد.

### 13. Error Handling

در صورت شکست ارسال Notification:

خطا باید ثبت شود.

وضعیت Notification باید قابل تشخیص باشد.

سیستم نباید باعث شکست عملیات اصلی Business شود، مگر در مواردی که Notification جزء ضروری همان عملیات باشد.

در صورت استفاده از Channel های خارجی، امکان Retry در معماری آینده باید قابل پشتیبانی باشد.

### 14. Logging & Audit

رویدادهای مهم Notification باید قابل ثبت و بررسی باشند.

حداقل موارد قابل ثبت:

ایجاد Notification

ارسال

Failure

Read



تغییر وضعیت

Channel مورد استفاده

## 15. UI/UX Requirements

Notification Center باید در طراحی Figma مشخص شده و مطابق UI/UX تأییدشده پیاده‌سازی شود.

حداقل عناصر UI:

Notification Icon

Unread Badge

Notification List

Notification Item

Read/Unread State

Timestamp

Notification Detail

Empty State

Loading State

Error State

طراحی باید Responsive باشد و در Desktop و Mobile قابل استفاده باشد.

## 16. MVP Scope

Included in MVP

In-App Notifications

Notification Center

Notification List

Unread Counter

Mark as Read

Mark All as Read

Notification Detail

Notification History

Notification Types

اتصال به رویدادهای اصلی سیستم

Authentication / Authorization

API

Database

Error Handling

Basic Logging

Conditional / Depending on Final Scope

Email

SMS

Notification Preferences

Future

Push Notification

Advanced Notification Preferences

Campaign Notifications

Scheduled Notifications

Digest Notifications

AI-generated Notifications

Personalized Notifications

Intelligent Notification Prioritization

Multi-channel Notification Orchestration

## **Acceptance Criteria .17**

ماژول زمانی قابل پذیرش است که:

- Notification برای User صحیح ایجاد شود.
- Notification در Notification Center نمایش داده شود.
- Unread Counter به درستی کار کند.
- کاربر بتواند Notification را Read کند.
- امکان Mark All as Read وجود داشته باشد.
- Notification Detail به درستی نمایش داده شود.
- Deep Link های تعریف شده صحیح کار کنند.

- کاربر به Notification سایر کاربران دسترسی نداشته باشد.
- API ها مطابق Scope نهایی کار کنند.
- Error Handling انجام شده باشد.
- تست های مرتبط با عملکرد اصلی انجام شده باشند.
- Integration با ماژول های مورد نیاز انجام شده باشد.
- نسخه نهایی روی محیط مورد نظر Deploy شود.
- Source Code و مستندات مربوط به ماژول تحویل شود.

## 18. Future Architecture Considerations

Notification Management باید به صورت Modular و قابل توسعه طراحی شود.

هدف این است که در نسخه های آینده بتوان بدون بازنویسی اساسی سیستم، موارد زیر را اضافه کرد:

Event → Notification Engine → Personalization → Channel Selection → Delivery → Tracking

و در مراحل پیشرفته تر:

AI → User Context → Priority → Best Channel → Best Time → Personalized Notification

این موارد جزء Scope اجرایی MVP نیستند و صرفاً به عنوان مسیر توسعه معماری آینده در نظر گرفته می شوند.

## 19. Dependencies

این ماژول به صورت مستقیم یا غیرمستقیم به موارد زیر وابسته است:

- Authentication
- User Management
- Seller Management
- Order Management
- Payment
- Product Management
- Admin Management
- API Layer
- Database

Dependency های دقیق بر اساس معماری نهایی پروژه و Scope تأیید شده مشخص خواهند شد.

## 20. Deliverables

Developer باید در پایان توسعه این ماژول موارد زیر را تحویل دهد:

- Source Code
- Database Changes / Migration
- API Implementation

- UI Integration
- Tests
- Error Handling
- Documentation
- Configuration
- Deployment
- Technical Notes در صورت نیاز

این مازول باید مطابق Definition of Done & Acceptance Criteria پروژه بررسی و پس از تأیید Product Owner به عنوان Completed/Accepted ثبت شود.

این ساختار با اصول متداول SRS همراستاست که SRS را به عنوان مبنای طراحی، پیاده سازی، تست و پذیرش نرم افزار تعریف می کنند.

## SRS-11 — Backend API & Frontend–Backend Integration

### Software Requirements Specification

#### 1. مشخصات سند

نام پروژه: Virashops

نام مازول: Backend API & Frontend–Backend Integration

کد SRS: SRS-11

وضعیت: MVP

نوع سند: Software Requirements Specification

مخاطبان: Product Owner ،QA ،Frontend Developer ،Backend Developer ،CTO

اولویت: Critical / Must Have

#### 2. هدف مازول

هدف این مازول ایجاد لایه استاندارد و امن ارتباطی بین Frontend و Backend ویرایش است.

این مازول باید امکان ارتباط قابل اتکا بین رابط کاربری و سرویس های Backend را فراهم کند و مسئولیت هایی مانند:

- API Design
- API Implementation

- Request / Response Handling
- Authentication
- Authorization
- Validation
- Error Handling
- Data Serialization
- API Security
- Integration
- Testing
- Documentation

را پوشش دهد.

اصل معماری:

Frontend → API Layer → Backend / Domain Services → Database / External Services

Frontend نباید مستقیماً به Database یا سرویس‌های داخلی Backend دسترسی داشته باشد.

### 3. Scope

در MVP این مازول شامل موارد زیر است:

1. API Architecture
2. REST API
3. Authentication API
4. Authorization
5. User API
6. Seller API
7. Product API
8. Category API
9. Cart API
10. Order API
11. Payment Integration API
12. Notification API
13. Admin API
14. Validation

- Error Handling .15
- API Security .16
- API Documentation .17
- Frontend Integration .18
- API Testing .19
- Logging .20
- Performance Considerations .21

#### **API Architecture .4**

API Layer باید به عنوان Contract بین Frontend و Backend عمل کند.

ساختار کلی:

Frontend



API / API Gateway



Application / Domain Services



Database / External Services

در صورت استفاده از معماری Microservices، API Layer نباید جزئیات داخلی سرویس‌ها را مستقیماً به Frontend expose کند.

#### **API Design Principles .5**

API ها باید:

- Consistent
- Predictable
- Secure
- Versionable
- Documented
- Testable
- Maintainable
- Scalable

باشند.

نامگذاری Endpoint ها، HTTP Method ها، Response ها و Error ها باید دارای استاندارد یکسان باشند.

## 6. API Versioning

API باید از Versioning پشتیبانی کند.

نمونه:

api/v1/.../

هدف این است که تغییرات آینده API بدون شکستن Frontend یا Client های قبلی قابل مدیریت باشند.

Versioning Strategy نهایی توسط CTO تعیین می شود.

## 7. HTTP Methods

از HTTP Method های استاندارد استفاده شود:

GET

برای دریافت اطلاعات.

POST

برای ایجاد Resource یا اجرای عملیات مشخص.

PUT / PATCH

برای Update.

DELETE

برای حذف یا Deactivate کردن Resource در موارد مجاز.

استفاده از Method ها باید با رفتار واقعی Endpoint مطابقت داشته باشد.

## 8. Request Structure

Request های API باید ساختار استاندارد داشته باشند.

موارد احتمالی:

Headers

Authentication Token

Path Parameters

Query Parameters

Request Body

Input باید قبل از ورود به Business Logic Validation شود.

## 9. Response Structure

Response های API باید دارای ساختار Consistent باشند.

نمونه مفهومی:

Success

Data

Metadata

Error

ساختار دقیق Response Contract باید در API Documentation مشخص شود.

## 10. HTTP Status Codes

سیستم باید از HTTP Status Code های استاندارد استفاده کند.

حداقل:

- Success 200
- Created 201
- No Content 204
- Bad Request 400
- Unauthorized 401
- Forbidden 403
- Not Found 404
- Conflict 409
- Validation Error 422
- Rate Limited 429



• Internal Server Error 500

استفاده از Status Code باید Consistent باشد.

## 11. Authentication API

API Layer باید Authentication سیستم را پشتیبانی کند.

موارد: MVP

Login

Logout

Session / Token Management

Authentication Validation

Refresh Token در صورت استفاده در معماری نهایی

Authentication باید براساس Architecture نهایی CTO پیاده‌سازی شود.

## 12. Authorization

Authentication به‌تنهایی کافی نیست.

هر Request حساس باید Authorization شود.

نمونه:

Customer

→ فقط اطلاعات مجاز خود را مشاهده کند.

Seller

→ فقط اطلاعات Seller مربوط به خود را مدیریت کند.

Admin

→ براساس Role و Permission دسترسی داشته باشد.

Authorization باید در Backend اعمال شود و نباید صرفاً به Frontend وابسته باشد.

## 13. User API

API های موردنیاز User Management باید امکان موارد زیر را فراهم کنند:

- User Registration در صورت وجود در Scope
- Login
- User Profile
- Update Profile
- User Status
- User Orders
- Authentication-related operations

دسترسی به اطلاعات باید براساس Permission انجام شود.

#### **Seller API .14**

API های Seller Management باید امکان:

- Seller Registration
- Seller Profile
- Seller Information
- Product Management
- Seller Status
- Seller Orders
- Seller Settings

را براساس Scope نهایی فراهم کنند.

#### **Product API .15**

Product API باید امکان موارد زیر را فراهم کند:

- Product List
- Product Detail
- Product Search
- Product Filter
- Product Creation در صورت مجاز بودن
- Product Update
- Product Status
- Category Relationship
- Seller Relationship

API باید قابلیت Pagination داشته باشد.

## Category API .16

Category API باید امکان:

- Category List
- Category Detail
- Parent / Child Category
- Category Filtering
- Admin Category Management برای

را فراهم کند.

## Search API .17

Search API باید امکان جستجوی کالا و موجودیت‌های موردنیاز MVP را فراهم کند.

حداقل:

Keyword Search

Category Filter

Basic Filters

Pagination

ساختار Search باید به‌گونه‌ای باشد که در آینده امکان اتصال به Search Engine تخصصی وجود داشته باشد.

## Cart API .18

Cart API باید امکان:

- ایجاد Cart
- مشاهده Cart
- Add Item
- Update Quantity
- Remove Item
- Clear Cart
- Calculate Cart Summary

را فراهم کند.

Cart نباید مسئول نهایی کردن Order باشد.

## Order API .19

Order API باید امکان:

- Create Order
- Order Detail
- Order List
- Order Status
- Order Items
- Customer Orders
- Seller Orders
- Cancellation در صورت مجاز بودن

را فراهم کند.

Order باید با Payment و Fulfillment قابل Integration باشد.

## Payment API Integration .20

Payment نباید مستقیماً توسط Frontend مدیریت شود.

Frontend باید درخواست Payment را از طریق API Backend ایجاد کند.

ساختار کلی:

Frontend

Create Payment Request →

Backend →

Payment Provider →

Payment Result / Callback →

Backend →

Update Payment / Order →

Frontend →

اطلاعات حساس Payment نباید در Frontend ذخیره یا پردازش غیرضروری شود.

## Notification API .21

API باید امکان:

- دریافت Notification ها
- دریافت Unread Count
- Mark as Read
- Mark All as Read
- Notification Detail

را فراهم کند.

## Admin API .22

Admin API باید برای Admin Panel امکانات لازم را فراهم کند.

مانند:

- User Management
- Seller Management
- Product Management
- Order Management
- Category Management
- Content Management
- System Settings

تمام Endpoint های Admin باید دارای Authorization مناسب باشند.

## Validation .23

تمام Input های API باید در Backend Validation شوند.

Validation باید شامل مواردی مانند:

Required Fields

Data Type

Format

Length

Range

Business Rules

باشد.

Frontend Validation جایگزین Backend Validation نیست.

## **.24 Error Handling**

تمام API ها باید Error Handling استاندارد داشته باشند.

Error Response نباید اطلاعات حساس مانند:

Stack Trace

Database Details

Internal Service Information

Secret Information

را به Client نمایش دهد.

Error باید دارای:

Error Code

Message

HTTP Status

Validation Details در صورت نیاز

Request / Correlation ID در صورت استفاده

باشد.

## **.25 Business Logic**

Business Logic نباید در Frontend قرار گیرد.

Frontend مسئول:

Presentation

User Interaction

Local UI State

است.

Backend مسئول:

Business Rules

Validation

Authorization

Pricing

Order Rules

Payment Rules

Seller Rules

Security

است.

## 26. Frontend Integration

Frontend باید از API Contract تعریف شده استفاده کند.

برای هر API باید مشخص باشد:

- Endpoint
- Method
- Request
- Response
- Error
- Authentication
- Loading State
- Empty State
- Retry Behavior

## 27. API Client Layer

Frontend باید دارای API Client / Service Layer مناسب باشد.

API Call ها نباید به صورت پراکنده در Component ها نوشته شوند.

ساختار پیشنهادی:

UI Component

↓

Frontend Service / API Client

↓

API

↓

Backend

این ساختار Maintainability و قابلیت تغییر API را افزایش می دهد.

## 28. Loading State

برای Request های Asynchronous باید Loading State وجود داشته باشد.

UI نباید هنگام دریافت اطلاعات در وضعیت نامشخص باقی بماند.

## 29. Empty State

در صورتی که API داده ای برنگرداند، Frontend باید Empty State مناسب نمایش دهد.

مثلاً:

No Orders Found

یا:

هنوز سفارشی ثبت نکرده اید.

## 30. Retry

برای خطاهای موقتی، امکان Retry باید در موارد مناسب وجود داشته باشد.

Retry نباید برای عملیات حساس مانند Payment بدون Idempotency و کنترل مناسب انجام شود.



### **Idempotency .31**

برای عملیات حساس مانند:

Payment

Order Creation

عملیات مالی

عملیات حساس Business

باید در صورت نیاز Idempotency در نظر گرفته شود.

هدف جلوگیری از ایجاد عملیات تکراری در اثر:

Double Click

Network Retry

Timeout

Duplicate Request

است.

### **Pagination .32**

API های List باید Pagination داشته باشند.

Pagination باید در Backend انجام شود.

حداقل اطلاعات موردنیاز:

Items

Page

Page Size

Total

Total Pages در صورت نیاز

### **Filtering & Sorting .33**

APIهای موردنیاز باید امکان:

Filter

Sort

Search

را داشته باشند.

پارامترها باید استاندارد و قابل توسعه باشند.

## **Security .34**

API Layer باید حداقل موارد زیر را رعایت کند:

Authentication

Authorization

Input Validation

Rate Limiting

Secure Headers

CORS Configuration

Protection Against Injection

Access Control

Secure Token Handling

Logging

Secret Managemen

## **Rate Limiting .35**

Endpointهای حساس باید Rate Limit مناسب داشته باشند.

خصوصاً:

Login

OTP

Password Reset

Search

Public API

Payment-related Endpoints

مقادیر دقیق Rate Limit توسط CTO تعیین می‌شود.

### **Caching .36**

در موارد مناسب API Layer می‌تواند از Caching استفاده کند.

مناسب برای:

Category

Product Data با تغییر کم

Public Configuration

Read-heavy Data

Caching نباید باعث نمایش داده‌های منقضی در عملیات حساس شود.

### **Logging .37**

Request‌های مهم API باید Logging مناسب داشته باشند.

اطلاعات حساس مانند Password، Token و اطلاعات محرمانه Payment نباید Log شوند.

### **Monitoring .38**

در معماری Production باید امکان Monitoring موارد زیر وجود داشته باشد:

- API Response Time
- Error Rate
- Request Count
- Failed Requests
- Service Health
- Database Performance

## 39. API Documentation

تمام API های Production باید مستند باشند.

Documentation حداقل باید شامل:

Endpoint

HTTP Method

Authentication

Parameters

Request Example

Response Example

Error Responses

Status Codes

باشد.

در صورت استفاده از REST، استفاده از OpenAPI / Swagger در نظر گرفته شود.

## 40. Testing

Unit Testing

برای:

Validation

Business Logic

API Services

Integration Testing

برای:

Frontend → API

API → Backend

Backend → Database

Payment Integration

End-to-End Testing

برای Flow های اصلی:

Login → Product → Cart → Order → Payment

و Flow های اصلی مرتبط با Seller و Admin.

#### **Frontend-Backend Contract .41**

Frontend و Backend باید براساس API Contract مشترک توسعه داده شوند.  
تغییر Breaking در API نباید بدون هماهنگی و ثبت Change Request انجام شود.

در صورت تغییر: Contract:

Documentation Update

Backend Update

Frontend Update

Test Update

باید انجام شود.

#### **API Compatibility .42**

API باید تا حد امکان Backward Compatible باشد.

تغییرات Breaking باید:

Versioned

Documented

Communicated

باشند.

#### **External Services .43**

در صورت استفاده از سرویس های خارجی، Integration باید در Backend یا Integration Layer مدیریت شود.

مانند:

- Payment Gateway
- SMS Provider
- Email Provider
- Storage
- Search Engine
- Maps / Location Services

Frontend نباید مستقیماً مسئول مدیریت Integration های حساس خارجی باشد.

#### 44. Scalability

API Architecture باید امکان رشد سیستم را فراهم کند.

در آینده باید امکان توسعه برای:

- Microservices
- API Gateway
- Async Processing
- Message Broker
- Caching
- Search Infrastructure
- Horizontal Scaling

وجود داشته باشد.

پیاده سازی تمام این موارد در MVP الزاماً ضروری نیست، مگر اینکه در Architecture نهایی پروژه قرار گرفته باشند.

#### 45. Future Architecture

معماری باید قابلیت حرکت از:

MVP Backend

به:

Modular / Scalable Backend

و در صورت نیاز:

Microservices Architecture

را داشته باشد.

Business Logic باید تا حد امکان از:

Frontend

UI

Framework-specific Components

جدا باشد.

## **Acceptance Criteria .46**

ماژول زمانی قابل پذیرش است که:

1. API Architecture مشخص و مستند باشد.
2. API های اصلی MVP پیاده سازی شده باشند.
3. Authentication صحیح باشد.
4. Authorization در Backend اعمال شود.
5. Validation کامل باشد.
6. Error Handling استاندارد باشد.
7. User API فعال باشد.
8. Seller API فعال باشد.
9. Product API فعال باشد.
10. Category API فعال باشد.
11. Cart API فعال باشد.
12. Order API فعال باشد.
13. Payment Integration مطابق Scope انجام شده باشد.
14. Notification API فعال باشد.
15. Admin API های مورد نیاز فعال باشند.
16. Pagination و Filter های مورد نیاز کار کنند.
17. Frontend به API های واقعی متصل باشد.
18. Loading / Empty / Error State ها مدیریت شده باشند.
19. API Documentation تکمیل باشد.
20. تست های مورد نیاز انجام شده باشند.
21. Security Review انجام شده باشد.

22. Deployment انجام شده باشد.

23. Product Owner Flow های اصلی را تأیید کرده باشد.

#### 47. Definition of Done

این مازول زمانی Done محسوب می شود که:

API Design → Implementation → Frontend Integration → Testing → Security Review → Documentation  
→ Deployment → Product Review → Acceptance

کامل شده باشد.

صرفاً ایجاد Endpoint یا اتصال موقت Frontend به Backend به عنوان تکمیل مازول محسوب نمی شود.

#### 48. Developer Deliverables

Developer باید تحویل دهد:

- Backend API Source Code
- Frontend API Integration
- Database Changes / Migration
- API Documentation
- OpenAPI / Swagger در صورت استفاده
- Unit Tests
- Integration Tests
- E2E Tests در Scope مربوطه
- Authentication / Authorization
- Error Handling
- Logging
- Configuration
- Deployment
- Technical Documentation
- Known Limitations

#### 49. Dependencies

این مازول با بخش های زیر ارتباط دارد:

- Authentication



- User Management
- Seller Management
- Product Catalog
- Search / Discovery
- Cart
- Order Management
- Payment
- Notification
- Admin Panel
- Database
- External Services

## **CTO Decisions Required .50**

موارد زیر باید توسط CTO در Architecture نهایی مشخص شوند:

1. REST یا GraphQL + REST
2. API Gateway مورد استفاده
3. API Versioning Strategy
4. Authentication Strategy
5. Token / Session Strategy
6. RBAC Architecture
7. Database Access Pattern
8. Validation Framework
9. Error Contract
10. Logging Strategy
11. Rate Limiting
12. Caching Strategy
13. API Documentation Standard
14. Testing Strategy
15. Deployment Architecture
16. Microservices Scope
17. Message Broker Scope

## 18. External Service Integration Pattern

هر مورد خارج از Scope تأییدشده باید طبق Change Request Procedure پروژه بررسی و تأیید شود.

## 51. Traceability

Requirements این SRS باید قابلیت Trace شدن به ساختار زیر را داشته باشند:

Feature → User Flow → Figma → API Contract → Backend Implementation → Database / Service →  
Frontend Integration → Test Case → Acceptance Criteria

هدف این است که هیچ Feature اصلی بدون مسیر مشخص از Requirement تا Implementation و Acceptance باقی نماند.

## SRS-12 — SEO & Content Foundation

زیرساخت SEO و محتوای MVP

### 1. هدف سند

این سند الزامات فنی و محتوایی موردنیاز برای ایجاد SEO & Content Foundation در نسخه MVP ویرایش را مشخص می‌کند.

هدف این است که معماری و صفحات MVP از ابتدا به شکلی پیاده‌سازی شوند که:

صفحات عمومی قابلیت Crawl و Index شدن داشته باشند.

ساختار URL منطقی و قابل توسعه باشد.

صفحات محصول و دسته‌بندی قابلیت دریافت Metadata اختصاصی داشته باشند.

زیرساخت لازم برای توسعه SEO در نسخه‌های بعدی وجود داشته باشد.

محتوای قابل ایندکس از محتوای خصوصی و غیرقابل ایندکس تفکیک شود.

توسعه آینده SEO بدون بازطراحی اساسی معماری امکان‌پذیر باشد.

این سند SEO Foundation فنی و محتوایی MVP است و شامل اجرای کامل استراتژی SEO، Content Marketing یا فعالیت‌های Off-page SEO نمی‌شود.

### 2. محدوده SRS

این SRS شامل موارد زیر است:

SEO Architecture

SEO-friendly URL Structure

Metadata Management  
Page Title Management  
Meta Description Management  
Canonical URL  
Robots Management  
Sitemap  
Structured Data Foundation  
Heading Structure  
Internal Linking Foundation  
Image SEO Foundation  
Index / No-Index Control  
SEO-ready Product Pages  
SEO-ready Category Pages  
SEO-ready Content Pages  
Content Management Foundation  
SEO Technical Validation

### 3. صفحات مشمول SEO در MVP

صفحات عمومی و قابل ایندکس MVP، در صورت تأیید Scope نهایی محصول، باید امکان SEO داشته باشند.

#### Homepage 3.1

Homepage باید دارای:

Title

Meta Description

Canonical URL

Heading Structure

قابل ایندکس بودن

Structured Data در صورت نیاز

باشد.

## Product Pages 3.2

هر صفحه محصول باید امکان تعریف و مدیریت موارد زیر را داشته باشد:

Product Name

SEO Title

Meta Description

Canonical URL

Product Description

Product Images

Image Alt Text

Category

Brand در صورت وجود

Price

Availability

Product Structured Data

اطلاعات SEO نباید صرفاً به صورت Hard-coded در Frontend قرار داده شوند.

## Category Pages 3.3

هر Category باید قابلیت داشتن:

Category Name

SEO Title

Meta Description

SEO Description

Canonical URL

Index / No-Index Status

Heading Structure

Internal Links

را داشته باشد.

ساختار Category باید به شکلی باشد که در آینده بتوان صفحات و زیرصفحات بیشتری به آن اضافه کرد.

#### Seller Pages 3.4

در صورت قرار گرفتن Seller Profile در Scope صفحات عمومی، ساختار SEO آن باید از ابتدا قابل توسعه باشد.

موارد قابل پشتیبانی:

Seller Name

Seller Description

Seller Location

Seller Products

Seller Rating در صورت مجاز بودن

SEO Title

Meta Description

Canonical URL

Index / No-Index

صفحات خصوصی Seller نباید به صورت تصادفی در موتورهای جستجو Index شوند.

#### Content Pages 3.5

سیستم باید امکان ایجاد صفحات محتوایی عمومی را داشته باشد.

نمونه:

About

FAQ

Guides

Buying Guides

Informational Pages

Landing Pages

سایر صفحات محتوایی موردنیاز MVP

#### URL Architecture .4

ساختار URL باید:

خوانا

پایدار

قابل توسعه

معنادار

بدون وابستگی به ID های نامفهوم تا حد امکان

باشد.

نمونه:

products/

products/category/

products/category/product-name/

sellers/seller-name/

content/article-name/

ساختار نهایی URL باید با معماری محصول و Routing سیستم هماهنگ شود.

URL های غیرضروری و Duplicate باید تا حد امکان کنترل شوند.

Google نیز استفاده از URL های توصیفی و ساختار منطقی صفحات را برای درک بهتر سایت توصیه می کند.

## 5. Metadata Management

سیستم باید امکان مدیریت Metadata را برای Entity های اصلی فراهم کند.

حداقل:

title

meta description

canonical

robots

Open Graph Metadata

Social Sharing Metadata

Metadata باید Dynamic باشد.

یعنی اطلاعات مربوط به هر Product، Category یا Content Page بتواند از Backend/CMS دریافت و در Frontend Render شود.

## 6. Title

هر صفحه عمومی باید Title اختصاصی داشته باشد.

قواعد:

عدم استفاده از Title یکسان برای صفحات مستقل

امکان تولید Dynamic Title

امکان Override کردن Title پیش فرض

محدود کردن ایجاد Title های خالی

پشتیبانی از Template های Title

## 7. Meta Description

برای صفحات اصلی قابل ایندکس، امکان تعریف Meta Description وجود داشته باشد.

سیستم باید بتواند:

Description اختصاصی دریافت کند.

Description پیش فرض تولید کند.

در صورت نیاز Description را Override کند.

Meta Description باید به صورت Page-specific مدیریت شود.

## 8. Canonical URL

صفحات عمومی باید امکان تعیین Canonical URL داشته باشند.

Canonical برای جلوگیری از مشکلات ناشی از:

Duplicate URL

Query Parameters

Filter Pages

Sorting

Pagination

صفحات مشابه

در معماری در نظر گرفته شود.

Google نیز Canonicalization را یکی از روش های مدیریت محتوای تکراری می داند.

## 9. Robots / Index Control

برای هر نوع Page باید امکان تعیین وضعیت Index وجود داشته باشد.

حداقل:

index

noindex

follow

nofollow

با توجه به نوع صفحه.

صفحات زیر معمولاً نباید بدون بررسی وارد Index شوند:

Cart

Checkout

Account

Login

Private Orders

Internal Search Results

صفحات خصوصی کاربران

صفحات دارای اطلاعات شخصی

صفحات موقت

صفحات سیستمی

برای جلوگیری از Index شدن باید از مکانیزم‌های مناسب مانند noindex و کنترل دسترسی استفاده شود؛ robots.txt به‌تنهایی ابزار مناسبی برای جلوگیری از Index شدن نیست.

## 10. Sitemap

سیستم باید قابلیت تولید Sitemap برای URL‌های عمومی قابل ایندکس را داشته باشد.

حداقل باید قابلیت پوشش:

Products

Categories

Sellers در صورت Public بودن

Content Pages

را داشته باشد.

Sitemap باید Dynamic باشد تا با اضافه‌شدن یا تغییر Entity‌ها قابل به‌روزرسانی باشد.



Google Sitemap را یکی از ابزارهای مهم برای کمک به کشف و اولویت‌بندی URLها می‌داند.

## 11. Structured Data Foundation

معماری باید امکان اضافه کردن Structured Data را فراهم کند.

برای MVP، بسته به صفحات نهایی، اولویت با موارد مرتبط با:

Product

Organization

Breadcrumb

WebSite

سایر Schemaهای موردنیاز صفحات

است.

Structured Data باید به صورت:

Dynamic

قابل توسعه

قابل نگهداری

هماهنگ با داده‌های واقعی سیستم

پیاده‌سازی شود.

اطلاعات Structured Data نباید با اطلاعات واقعی نمایش داده شده به کاربر مغایرت داشته باشد.

Structured Data می‌تواند به موتور جستجو در درک بهتر محتوای صفحه و در برخی موارد ایجاد Search Features کمک کند.

## 12. Heading Structure

صفحات باید ساختار Heading منطقی داشته باشند.

حداقل:

H1

H2

H3

...

استفاده از Heading باید بر اساس ساختار معنایی صفحه باشد، نه صرفاً برای تنظیم ظاهر UI.

### 13. Internal Linking Foundation

معماری باید امکان ایجاد Internal Linking مناسب را فراهم کند.

نمونه:

Homepage



Category



Subcategory



Product

و در صورت نیاز:

Content



Category



Product

Internal Linkها باید تا حد امکان Crawlable باشند. Google استفاده از لینک‌های قابل Crawl را برای کشف صفحات توصیه می‌کند.

### 14. Image SEO

برای تصاویر اصلی Product و Content باید امکان مدیریت موارد زیر وجود داشته باشد:

Alt Text

File Name

Image URL

Image Dimensions

مناسب بودن Format

Lazy Loading در موارد مناسب

جلوگیری از تصاویر غیرضروری یا بیش از حد سنگین

Alt Text باید بر اساس محتوای واقعی تصویر ایجاد شود.

## 15. Content Management Foundation

سیستم باید حداقل زیرساخت لازم برای مدیریت محتوای MVP را داشته باشد.

Content Entity می‌تواند شامل:

Title

Slug

Body

Featured Image

SEO Title

Meta Description

Canonical

Status

Publish Date

Update Date

Author در صورت نیاز

باشد.

Status می‌تواند شامل:

Draft

Published

Archived

باشد.

## 16. Slug Management

Entityهای عمومی مانند:

Product

Category

Content

Seller در صورت Public بودن

باید Slug قابل مدیریت داشته باشند.

Slug باید:

Unique

Stable

URL-safe

قابل تغییر با کنترل مناسب

باشد.

در صورت تغییر Slug، امکان مدیریت Redirect باید در معماری آینده پیش‌بینی شود.

## 17. SEO Rendering

با توجه به معماری تخصصی جدید ویرایش و استفاده از Frontend مدرن، صفحات Public باید به شکلی Render شوند که محتوای اصلی و Metadata برای Crawler قابل دسترسی باشد.

صفحات مهم SEO نباید صرفاً به Client-side Rendering وابسته باشند، به‌گونه‌ای که محتوای اصلی بدون اجرای JavaScript قابل دسترسی نباشد.

برای صفحات عمومی، روش Rendering باید با نیاز SEO و معماری Next.js هماهنگ شود.

Google برای JavaScript-based applications توصیه می‌کند URL‌های مستقل برای محتوا وجود داشته باشند و صفحات به شکلی ساخته شوند که Crawler بتواند محتوای آنها را پیدا و Render کند.

## 18. Search & Filter SEO

Search و Filterها باید از ابتدا با SEO تفکیک شوند.

هر URL فیلترشده الزاماً نباید Index شود.

Developer باید برای مواردی مانند:

Search Query

Sorting

Filters

Pagination

Faceted Navigation

قواعد مشخصی برای:

Index

Noindex

Canonical

URL Parameters

در نظر بگیرد.

هدف جلوگیری از ایجاد حجم زیادی URL کم ارزش یا Duplicate است.

## 19. Performance Requirements مرتبط با SEO

در معماری Frontend باید موارد زیر در نظر گرفته شوند:

Fast Page Load

Optimized Images

مناسب بودن Rendering Strategy

کاهش JavaScript غیرضروری

Caching مناسب

جلوگیری از Blocking Resources

Responsive بودن

هدف این است که SEO به صورت جدا از Performance طراحی نشود.

## 20. SEO Data Model

معماری Backend/CMS باید امکان ذخیره اطلاعات SEO را برای Entity های مورد نیاز داشته باشد.

نمونه مفهومی:

Entity

title —|

slug —|

seoTitle —|

metaDescription —|

canonicalUrl —|

robotsIndex —|

robotsFollow —|

ogTitle —|

ogDescription —|

این ساختار باید Modular باشد تا در آینده Entity های بیشتری بتوانند SEO Metadata داشته باشند.

## 21. Admin / CMS Requirements

در پنل مدیریت، در صورت قرار گرفتن در Scope MVP، امکان مدیریت SEO برای Entity های اصلی باید وجود داشته باشد.  
حداقل:

SEO Title

Meta Description

Slug

Canonical

Index/Noindex

Open Graph

SEO Preview در صورت امکان

همچنین سیستم باید از ایجاد Metadata نامعتبر یا خالی برای صفحات مهم تا حد امکان جلوگیری کند.

## 22. Security & Privacy

اطلاعات خصوصی کاربران نباید وارد صفحات Public یا Search Index شوند.

موارد حساس مانند:

اطلاعات شخصی

سفارش های خصوصی

اطلاعات پرداخت

اطلاعات حساب

داده های خصوصی Seller/Buyer

نباید در صفحات عمومی قابل Crawl باشند.

## 23. SEO Technical Validation

قبل از Acceptance، موارد زیر باید بررسی شوند:

Crawlability

Public Pages قابل Crawl باشند.

لینک‌های اصلی قابل Crawl باشند.

منابع ضروری Block نشده باشند.

Indexability

صفحات موردنظر قابلیت Index داشته باشند.

صفحات Private یا غیرضروری Noindex باشند.

Metadata

Title وجود داشته باشد.

Meta Description در صفحات موردنیاز وجود داشته باشد.

Canonical صحیح باشد.

Sitemap

Sitemap تولید شود.

URLهای صحیح در Sitemap باشند.

URLهای Private در Sitemap نباشند.

Structured Data

Structured Data معتبر باشد.

با محتوای واقعی صفحه تطابق داشته باشد.

URL

URLها خوانا باشند.

Duplicate URLهای غیرضروری کنترل شوند.

Mobile

صفحات Public در Mobile قابل استفاده باشند.

Google برای واجدشرایط‌بودن یک صفحه جهت Search حداقل به دسترسی Crawler، پاسخ موفق HTTP و وجود محتوای قابل ایندکس نیاز دارد؛ بنابراین این موارد باید در Acceptance بررسی شوند.

## 24. MVP Acceptance Criteria

SRS-12 زمانی قابل قبول است که حداقل موارد زیر عملیاتی باشند:

[ ] SEO Metadata برای صفحات اصلی قابل مدیریت باشد.

[ ] Product SEO قابل مدیریت باشد.

[ ] Category SEO قابل مدیریت باشد.

- [ ] Content SEO قابل مدیریت باشد.
- [ ] URL Structure مشخص و پایدار باشد.
- [ ] Slug Management وجود داشته باشد.
- [ ] Canonical قابل مدیریت باشد.
- [ ] Index/Noindex قابل مدیریت باشد.
- [ ] Sitemap فعال باشد.
- [ ] Robots Configuration وجود داشته باشد.
- [ ] Structured Data Foundation وجود داشته باشد.
- [ ] Heading Structure صحیح باشد.
- [ ] Internal Linking Foundation رعایت شده باشد.
- [ ] Image SEO Foundation رعایت شده باشد.
- [ ] صفحات Private از Index شدن محافظت شوند.
- [ ] SEO Rendering با معماری Frontend سازگار باشد.
- [ ] Search/Filter URLs دارای Strategy مشخص باشند.
- [ ] SEO Technical Validation انجام شده باشد.

25. موارد خارج از Scope MVP

موارد زیر جزء این SRS نیستند، مگر اینکه در Scope رسمی پروژه به صورت جداگانه اضافه شوند:

- Full SEO Campaign
- Keyword Research کامل
- Content Marketing Campaign
- Link Building
- Backlink Acquisition
- Off-page SEO
- SEO Reporting پیشرفته
- Enterprise SEO
- AI SEO Automation
- AI Content Generation
- Advanced SEO Intelligence



Automated Price/SEO Intelligence

کامل International SEO

کامل Multi-language SEO

این موارد می‌توانند در نسخه‌های بعدی به عنوان Feature یا Module مستقل اضافه شوند.

Future Extensibility .26

معماری SRS-12 باید امکان توسعه آینده به سمت موارد زیر را داشته باشد:

Advanced SEO Analytics

AI SEO Assistant

AI Content Assistant

Automated Internal Linking

SEO Recommendations

Dynamic Landing Pages

Programmatic SEO

Advanced Structured Data

Multi-language SEO

Regional SEO

SEO Performance Dashboard

Search Intelligence

AI-powered Content Optimization

این قابلیت‌ها در MVP پیاده‌سازی نمی‌شوند؛ صرفاً معماری نباید مانع اضافه‌شدن آنها در آینده شود.

Developer Deliverables .27

Developer باید تحویل دهد:

SEO-ready Frontend Architecture

SEO Metadata Integration

URL/Slug Implementation

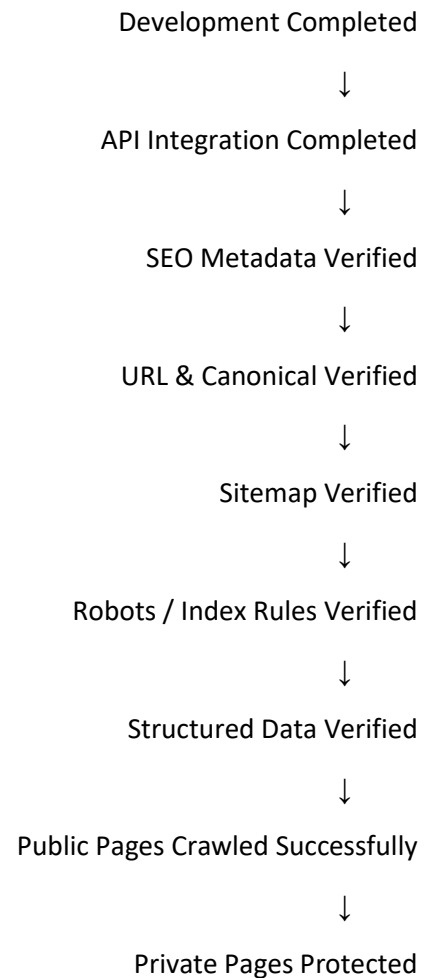
Canonical Implementation

Robots/Index Control

Sitemap Implementation  
Structured Data Foundation  
Product SEO Integration  
Category SEO Integration  
Content SEO Integration  
Image SEO Foundation  
SEO-related API Integration  
SEO Technical Documentation  
SEO Validation Results

Definition of Done .28

SRS-12 زمانی Done محسوب می‌شود که:





Mobile & Desktop Verified



Technical SEO Review



Product Owner Review



Acceptance

— SRS-12 — SEO & Content Foundation — زیرساخت SEO و محتوای MVP

SRS-13 — Security, QA & Deployment Foundation

زیرساخت امنیت، تضمین کیفیت و استقرار MVP

1. هدف سند

این سند الزامات پایه موردنیاز برای Security، Quality Assurance و Deployment نسخه MVP ویرایش را مشخص می‌کند.

هدف این SRS ایجاد یک Foundation قابل اتکا برای اجرای MVP است؛ به‌گونه‌ای که سیستم از ابتدا دارای حداقل استانداردهای امنیتی، تست، کنترل کیفیت، مدیریت خطا و استقرار باشد و در آینده امکان توسعه و مقیاس‌پذیری آن وجود داشته باشد.

این سند شامل پیاده‌سازی کامل Security Operations، DevSecOps، مانیتورینگ Enterprise و زیرساخت Cloud پیشرفته نیست و این موارد در صورت نیاز می‌توانند در مراحل بعد توسعه پیدا کنند.

2. محدوده SRS

این SRS شامل موارد زیر است:

Authentication Security

Authorization Security

API Security

Data Protection

Input Validation

Error Handling

Secure File Handling

Password & Credential Security

Environment & Secret Management

Logging Foundation

Testing Foundation

Unit Testing

Integration Testing

API Testing

Regression Testing

Build Validation

Deployment Foundation

Environment Management

CI/CD Foundation

Backup & Recovery Foundation

Production Readiness

### Security Architecture .3

امنیت باید به صورت Layered Security در سیستم پیاده‌سازی شود.

حداقل لایه‌ها:

Client



Frontend Security



API Gateway / API Layer



Authentication



Authorization



Business Logic



Database



Storage

هیچ Client نباید مستقیماً به Database یا Storage خصوصی دسترسی داشته باشد.

#### 4. Authentication

سیستم باید دارای مکانیزم Authentication امن باشد.

حداقل موارد:

Login

Logout

Session Management

Token Management

Password Management

Authentication State

Protected Routes

Protected API Endpoints

در صورت استفاده از JWT یا Session-based Authentication، نحوه مدیریت Token/Session باید در Architecture مستند شود.

#### 5. Authorization

Authentication به تنهایی کافی نیست.

سیستم باید دسترسی کاربران را بر اساس Role و Permission کنترل کند.

نمونه:

Buyer

Seller

Visitor

Admin

Manager

Support

سطح دسترسی باید در Backend نیز بررسی شود.  
Frontend UI نباید مرجع نهایی Authorization باشد.

## 6. API Security

تمام API های حساس باید دارای کنترل دسترسی مناسب باشند.  
موارد مورد نیاز:

Authentication Validation

Authorization Validation

Input Validation

Rate Limiting در نقاط حساس

Request Validation

Secure Error Response

جلوگیری از دسترسی غیرمجاز

جلوگیری از Mass Assignment

کنترل Resource Access

API نباید اطلاعات حساس غیرضروری را در Response برگرداند.

## 7. Input Validation

تمام ورودی های دریافت شده از:

User

Frontend

API

File Upload

Query Parameters

Request Body

Headers

باید Validate شوند.

Validation باید در Backend انجام شود و صرفاً به Validation سمت Frontend وابسته نباشد.

## 8. Data Protection

اطلاعات حساس باید با توجه به نوع داده محافظت شوند.

شامل:

Password

Authentication Data

Personal Information

Payment-related Information

Seller Information

Buyer Information

Private Orders

Internal System Data

Password ها نباید به صورت Plain Text ذخیره شوند.

## 9. Password Security

در صورت استفاده از Password Authentication:

Password باید Hash شود.

Hash Algorithm مناسب استفاده شود.

Password در Log ذخیره نشود.

Password در API Response برگردانده نشود.

Credential های حساس در Source Code قرار نگیرند.

## 10. Secret Management

موارد زیر نباید در Repository عمومی یا Source Code Hard-code شوند:

Database Credentials

JWT Secrets

API Keys

Payment Credentials

Storage Credentials

Third-party Credentials

## Production Secrets

Environment Variables یا Secret Management مناسب باید استفاده شود.

## 11. File Upload Security

در صورت وجود Upload:

سیستم باید موارد زیر را کنترل کند:

File Type

File Size

File Extension

MIME Type

Storage Location

Access Permission

File Naming

فایل‌های خصوصی نباید بدون Authorization قابل دسترسی عمومی باشند.

## 12. Error Handling

Error Handling باید استاندارد و قابل پیش‌بینی باشد.

در Production:

اطلاعات حساس نباید در Error Response نمایش داده شود.

Stack Trace نباید به User نمایش داده شود.

Error Code های مشخص استفاده شود.

Error ها در صورت نیاز Log شوند.

خطاهای Backend و Frontend قابل Trace باشند.

## 13. Logging Foundation

سیستم باید حداقل Logging موردنیاز برای Debug و بررسی خطاها را داشته باشد.

موارد قابل Log:

Authentication Events

Authorization Failures



API Errors

System Errors

Important Business Events

Deployment Events

موارد حساس مانند Password، Token و اطلاعات محرمانه نباید در Log ثبت شوند.

#### 14. Audit Foundation

برای عملیات حساس، معماری باید قابلیت ایجاد Audit Trail را در نظر بگیرد.

نمونه:

تغییر اطلاعات Seller

تغییر Product

تغییر Price

تغییر Order Status

تغییر Permission

عملیات Admin

Audit Log می‌تواند در MVP محدود باشد ولی معماری نباید مانع توسعه آن در آینده شود.

#### 15. Testing Strategy

Testing باید بخشی از فرآیند Development باشد، نه مرحله‌ای جداگانه در انتهای پروژه.

فرآیند:

Development

↓

Unit Test

↓

Integration Test

↓

API Test

↓

Regression Test



Review



Deployment

## Unit Testing .16

برای Business Logic و اجزای حساس سیستم باید Unit Test نوشته شود.  
اولویت با:

Core Business Logic

Validation

Authentication Logic

Authorization Logic

Pricing Logic

Order Logic

Payment Logic

مهمترین Services

است.

## Integration Testing .17

Integration Test باید ارتباط بین اجزای اصلی سیستم را بررسی کند.  
نمونه:

Frontend



API



Backend Service



Database

همچنین Integration های حساس مانند:

Authentication

Orders

Payment

Storage

Notification

در صورت وجود در Scope باید بررسی شوند.

## 18. API Testing

API های اصلی باید تست شوند.

حداقل بررسی:

Success Response

Validation Error

Unauthorized Request

Forbidden Request

Invalid Input

Missing Data

Not Found

Server Error

## 19. Regression Testing

پس از تغییرات مهم، قابلیت های قبلی نباید بدون بررسی مجدد وارد Production شوند.

Regression Testing باید حداقل برای Core Flow های MVP انجام شود.

## 20. Frontend QA

صفحات اصلی باید از نظر موارد زیر بررسی شوند:

Functional Correctness

Responsive Behavior

Form Validation

API Integration

Loading State

Empty State

Error State

Authentication State

Permission State

## 21. Build Validation

قبل از Deployment باید حداقل موارد زیر بررسی شوند:

Install



Build



Type Check



Lint



Tests



Production Build

در صورت وجود خطای جدی، Deployment نباید انجام شود.

## 22. Environment Management

حداقل Environment های زیر باید در معماری در نظر گرفته شوند:

Development

Staging

Production

در صورت محدودیت MVP می توان تعداد Environment ها را کاهش داد، اما Production باید از Development جدا باشد.

## 23. Configuration Management

Configuration های محیط های مختلف نباید در Source Code ثابت شوند.

نمونه:

Database URL

API URL

Storage Configuration

Authentication Configuration

Payment Configuration

Third-party Services

باید Environment-specific باشند.

## 24. CI/CD Foundation

پروژه باید قابلیت ایجاد Pipeline برای:

Code Push

↓

Install Dependencies

↓

Lint

↓

Type Check

↓

Test

↓

Build

↓

Deploy

را داشته باشد.

در MVP می توان Pipeline را ساده نگه داشت، اما ساختار باید قابل توسعه باشد.

## 25. Git & Branch Protection

Repository باید دارای Workflow مشخص باشد.

حداقل:

Feature Branch



Pull Request



Review



Merge



Build/Test



Deploy

استفاده مستقیم و کنترل نشده از Production Branch نباید روش اصلی توسعه باشد.

## 26. Deployment

Deployment باید Repeatable و مستند باشد.

Developer باید مشخص کند:

Application چگونه Build می شود.

Environment Variables چگونه تنظیم می شوند.

Database چگونه Migration می شود.

Application چگونه Start می شود.

Rollback چگونه انجام می شود.

## 27. Database Migration

تغییرات Database باید از طریق Migration یا فرآیند کنترل شده انجام شوند.

نباید Database Production به صورت دستی و بدون ثبت تغییرات اصلاح شود.

Migration ها باید:

Versioned

قابل پیگیری

قابل اجرا

مستند

باشند.

## Backup Foundation .28

برای داده‌های مهم باید Backup Strategy وجود داشته باشد.

حداقل:

Database Backup

Storage Backup در صورت نیاز

Backup Retention

Restore Procedure

در MVP می‌تواند ساده باشد، اما باید مشخص باشد که در صورت خرابی چگونه Recovery انجام خواهد شد.

## Recovery Foundation .29

سیستم باید حداقل یک فرآیند مشخص برای Recovery داشته باشد.

سناریوهای پایه:

Database Failure

Deployment Failure

Application Failure

Configuration Error

Data Loss

## Security Review .30

قبل از Acceptance حداقل بررسی امنیتی باید انجام شود.

موارد:

Authentication

Authorization

API Access

Input Validation

Secrets

File Upload

Error Handling

Sensitive Data Exposure

Permission Boundaries